

iViewNG™ SDK

v3.1

December 2012

Contents

1 iViewNG API **1**

1.1 Introduction 1

2 Concepts **2**

2.1 Architecture 2

2.2 Asynchronous Communication 3

3 Tutorial **7**

3.1 Step 1: Checking the compiler 7

3.2 Step 2: Starting a Server and Connecting 8

3.3 Step 3: Receiving Data Through the Callback 10

3.4 Step 4: Using the iViewNG Convenience Library 13

3.5 Step 5: Receiving Gaze Data 13

3.6 Calibration 17

4 Function Documentation **18**

4.1 iViewNG-BasicDatatypes.h File Reference 18

4.2 iViewNG-Calibration.h File Reference 19

4.3 iViewNG-Classified.h File Reference 27

4.4 iViewNG-Connection.h File Reference 29

4.5 iViewNG-Convenience.h File Reference 34

4.6 iViewNG-Core.h File Reference 34

4.7 iViewNG-DataAcquisition.h File Reference 48

4.8 iViewNG-DataStreams.h File Reference 53

4.9 iViewNG-Datatypes-Classified.h File Reference 65

4.10 iViewNG-Device-ETG-Classified.h File Reference 66

4.11 iViewNG-Device-ETG.h File Reference 74

© 2012 SensoMotoric Instruments GmbH

4.12 iViewNG-Device-Legacy.h File Reference 79
4.13 iViewNG-Device-RED5xx.h File Reference 79
4.14 iViewNG-Device-REDm.h File Reference 79
4.15 iViewNG-Device.h File Reference 80
4.16 iViewNG-Utility.h File Reference 85
4.17 iViewNG.h File Reference 89
4.18 D:/Research/iViewNG/Implementation/SMI/source/iViewNGLib/Api/iViewNG-SDK/iView-NG-SDK-Documentation.dox File Reference 91

Index **91**

Chapter 1

iViewNG API

1.1 Introduction

This is the documentation of the iView API 4.x, also referred to as iViewNG (iView Next Generation) API. The API was tailored to fit the next generation of iView. The basic functionality of the iViewX API 3.x is available as well, but the syntax and communication technique changed.

Some of the advantages are:

- Functions separated by field of application, no longer one big bunch of functions.
- Asynchronous, thread-safe communication.
- Low latency.
- Improved extendability.
- Virtually any number of clients can connect to one iViewNG server and operate simultaneously, receive individually configured data.
- Multiple operating systems are fully supported (e.g. Windows, Linux, Android).

This documentation will give you a short tour through all the used techniques and functions sections and provide short examples for understanding the new API.

- Concepts
- Tutorial

Chapter 2

Concepts

2.1 Architecture

The iViewNG API enables a client application to communicate with an iViewNG server which controls one or more eye tracking devices. Please note that devices which are operated by an iView-X server are not able to provide this communication scheme. They must be controlled using the iView-X API 3.x.

The API provides transparent communication to the iViewNG server. Internally, the communication is realized by a communication layer which supports the IP based communication protocols TCP and UDP as well as shared memory communication in case that client and server applications are running on the same machine. If required, the client application is in full control over the used communication technique.

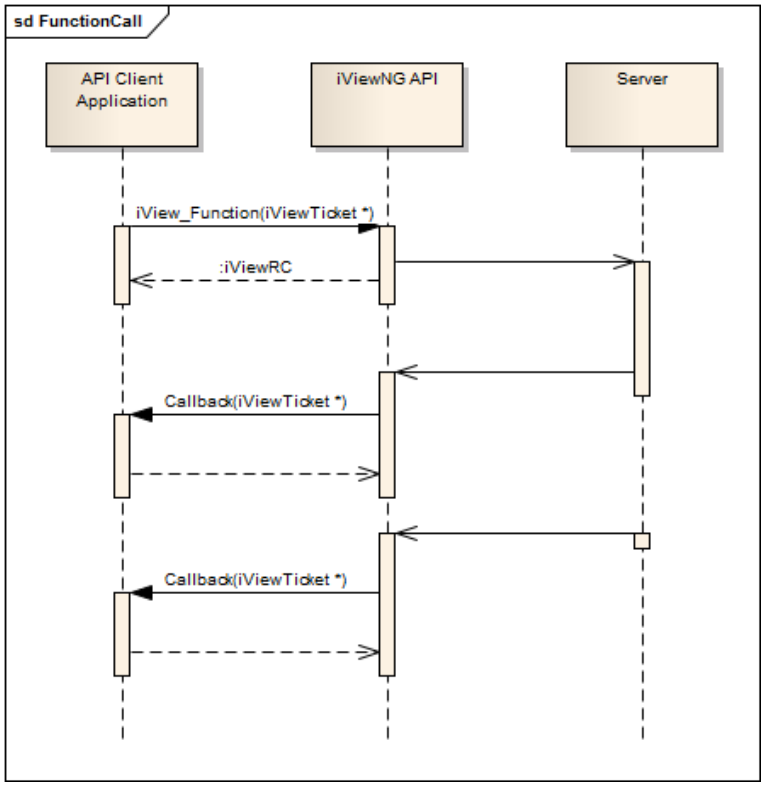


Figure 2.1: Caption

While the iViewNG API offers the core communication to control an eye tracking device and provides access to the acquired and calculated data, higher level operations are part of the SDK. That means they use the API like the client application to realize convenient operations. Examples for such operations are: automatic calibration, validation, display of various monitors.

2.2 Asynchronous Communication

All iViewNG API functions which invoke a server command operate asynchronously. That means, the API function does not block the client program until the command is processed by the server. Instead, the API backend returns the program flow to the caller immediately after briefly checking the command request and forwarding the request to the iViewNG server. A return code indicates if this brief check was successful: all command functions check if the API was initialized properly, if a connection to the server is available and if the command parameters seem to be plausible.

If you want your commands to behave like synchronous calls, have a look at the functions `iView_WaitForTicket()` and `iView_WaitForDataStream()` which are part of the iViewNG Convenience Library which is available as source code. These functions provide the functionality to block the control flow until a ticket is answered or until a data stream provides a data entity.

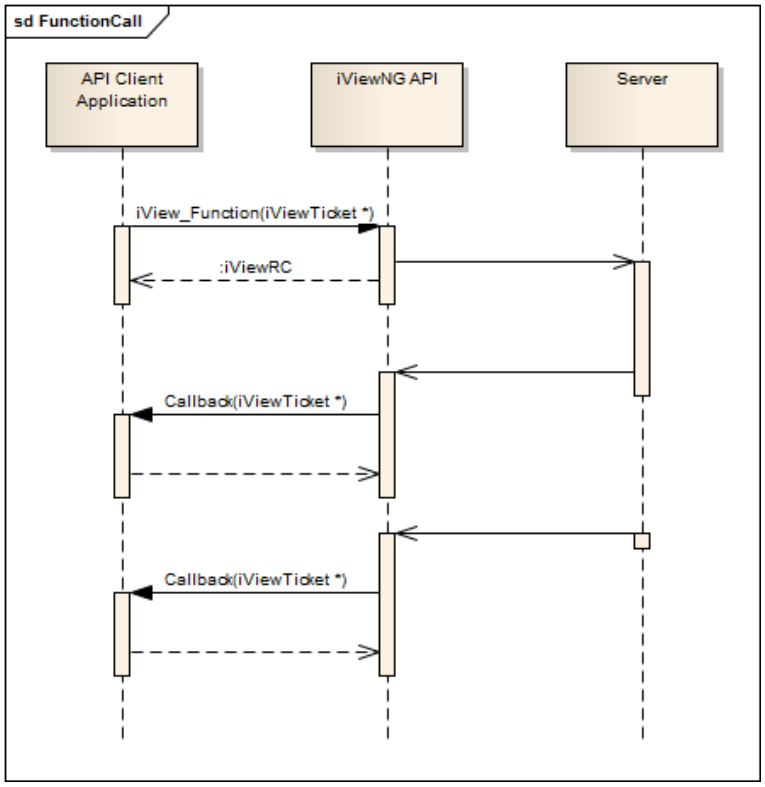


Figure 2.2: Caption

As soon as the command is processed by the iViewNG server, a message event is sent to the API client by calling a callback function implemented by the API user. This event includes the result of the command. For some commands, the callback will be called several times with each event providing a new result (data streams).

The API functions are thread-safe. All command functions use mutual exclusions to prevent other threads to operate until the first called function completed.

iViewTicket data structure

The ticket mechanism offer a way to associate an incoming message event with the command which the event belongs to. If the API user calls a command function, a ticket must be provided as well. Whenever the iViewNG server passes a message event to the user callback, the ticket which created the command is provided as parameter of the callback.

When a command is completed, iViewNG updates iViewTicket.state, iViewTicket.progress, iViewTicket.-returnCode and maybe iViewTicket.result. Thus, it is easily possible to find out which command created the incoming event as well as determining the processing state.

There are multiple ways to associate the incoming message event with the command which is answered:

1. comparing the ticket's address received by the callback with the adresses of all created tickets,
2. comparing the (unique) ticket's number with the numbers of all created tickets,

3. comparing the ticket result type which corresponds with the API command.

The ticket data structure contains the following attributes:

- iViewTicket.ticketNumber: a running number which is assigned to the ticket structure by iView_CreateTicket(). There will never be more than one ticket with this ticket number.
- iViewTicket.functionName: a string describing the function call which this ticket was used for, assigned to the ticket structure by iView_CreateTicket(). There may be more than one ticket with this function name. Use this only for debugging purposes.
- iViewTicket.state: the processing state of the command. The caller may at any time check if the request was already answered by examining the state indicator. If the server updates the state, the callback function will be called to announce the change.
- iViewTicket.progress: a progress indicator to let the user have an idea of how much of the command is already finished. Not all commands provide information about the progress, see the functions documentation. If the server updates the progress, the callback function will be called to announce the change.
- iViewTicket.returnCode: a return code to indicate the (intermediate) result of the command. Defaults to RC_NO_ERROR. A value different to RC_NO_ERROR indicates that iViewTicket.result points to an iViewServerError data structure if not NULL. Otherwise, iViewTicket.result points to a iViewResult data structure if not NULL. See functions documentations for details.
- iViewTicket.result: points to a data structure which includes the result of the operation. This may be an iViewServerError structure if the command raised an error or an iViewResult structure. Use iViewTicket.returnCode to determine if the result is an error or not and use typecasts.

The caller of the API functions is responsible for explicitly indicating to release all data structures! iViewNG will never release the iViewTicket structure if not ordered explicitly by the API user. The result is released only if:

- the API user orders iViewNG to release a ticket's result and uses the function iView_ReleaseResult(),
- the API user releases the ticket structure by using the function iView_ReleaseTicket(),
- the API user terminates the API by using iView_Shutdown() or
- a new data entity replaces the old result, which happens only with data streams.

Never try to release a ticket without calling iView_ReleaseTicket()! Never write into the ticket data structure!

The callback function

Whenever some data is submitted by the iViewNG server, the callback function is called. The callback function is a function the user must implement and register to the API. There is only one callback function for all incoming types of data.

See this example for a example callback function:

```
void MyCallback (iViewTicket * const ticket) {

    // this should never happen
    if (NULL == ticket)
        return;

    printf("Received ticket #%u with return code %u.\n", ticket->ticketNumber
      , ticket->returnCode);

    return;
}
```

Using the function iView_SetCallback(), this user-defined function is registered to the API.

There is only one callback call active at the same time. Should it happen that a new message event arrives while the callback function is still busy with processing the last one, the new message event will be queued and wait until the callback function is left. This is implemented thread-safe.

Therefor, it is important to keep the time spent inside the callback function as short as possible. For example, if you expect data to arrive with an average sample rate of 100 Hz, the time spent inside the callback function must not be larger or close to 10 milliseconds. Otherwise, the queue size would increase over time. For a tight timing, it is most essential to let the callback finish as fast as possible!

Please note, while the client application is busy with executing the callback function, no other API calls or anwers will be processed. It is not possible to use any API command function but iView_ReleaseResult() from inside the callback since this would create a deadlock!

Handling data streams

Data streams use a single ticket structure to provide a large number of data structures to the API user. For example, gaze samples are delivered periodically with approximately the eye camera's sample rate. The user-defined callback is called once for each incoming gaze sample, always using the same ticket structure. With each call, the ticket's result pointer iViewTicket.result will point to a new iViewResult structure which contains the new sample.

Subsequent message events overwrite the predecessing result. If a result is not released explicitly with iView_Result(), iViewNG will release the result data before creating a new result and storing the address in the ticket.

In case that server-generated data must exist outside the callback, e.g. image data which is passed to a user interface for display, it is required to copy the data inside the callback function.

Never try to release result data without calling iView_ReleaseTicket() or iView_ReleaseResult()! Never write into the result data structure!

© 2012 SensoMotoric Instruments GmbH

Chapter 3

Tutorial

This section explains the usage of the iViewNG API by example. Please note that the example source code is included as part of the iViewNG SDK package.

3.1 Step 1: Checking the compiler

Goal of this example is to make sure that your compiler tool chain is set up correctly, that the iViewNG SDK include files are found, that the iViewNG API library is found and that your runtime environment is able to execute your first program which uses the iViewNG SDK.

The source code of the example discussed in this section can be found in the SDK inside the folder *Tutorial-GetLibraryVersion*.

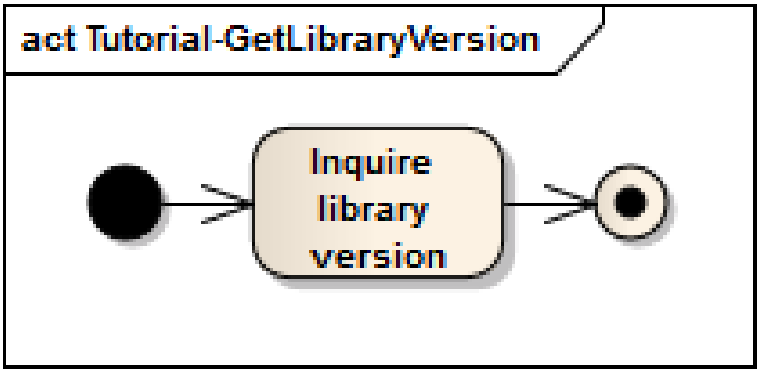


Figure 3.1: Caption

To check if the compiler and linker are up and running, it is recommended to use the example program *Example-GetVersion.cpp* which calls a simple API function:

```
#include <stdio.h>
#include <iViewNG-Core.h>

int main(int argc, char ** args) {
```

```
  iViewVersion version;

  iViewRC rc = iView_GetLibraryVersion(&version);

  if (RC_NO_ERROR != rc)
    printf("ERROR returned by iView_GetLibraryVersion(): %d\n", rc);

  printf( "The version of the libiViewNG is: %u.%u.%u.%u\n",
          version.major, version.minor, version.patch, version.
      build );

  return 0;
}
```

Please make sure that this program compiles without errors or warnings and that the created program displays the version of the iViewNG API library:

Note

For subsequent usages of the API functions, it is most convenient to encapsulate the function call as well as the check if the function call returned RC_NO_ERROR into a preprocessor macro:

```
// macro that test the return of an api call and outputs an error if necessary
#define CALL_API(_fun) { \
  if ( RC_NO_ERROR != (rc = _fun) ) { \
    View_iViewRcToString (&gErrorString, rc); \
    fprintf (stderr, "ERROR in '" #_fun "': %ls\r\n", gErrorString); \
    free(gErrorString); \
    return rc; \
  }\
}
extern wchar_t * gErrorString;

// the variable to hold the error return code is global here for convenience
extern iViewRC rc;
```

Note

This enhances the readability of the source code and asserts that errors are detected immediately. All other code examples will use this preprocessor macro.

3.2 Step 2: Starting a Server and Connecting

Goal of this example is that you understand the required steps to connect to a server:

- starting an iViewNG server process,
- connecting to it,
- submitting your license key. For these steps, you will learn how to use the ticket mechanism.

The source code of the example discussed in this section can be found in the SDK inside the folder *Tutorial-Connect*.

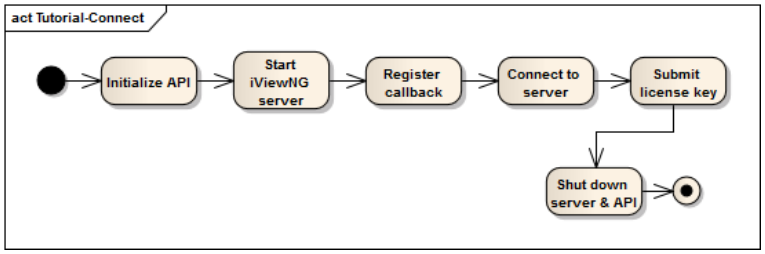


Figure 3.2: Caption

The client application may use the function iView_StartServer() to spawn an iViewNG server process. The process will be executed on the computer which runs the client application:

```
#include <stdio.h>
#include <iViewNG-Core.h>
#include <iViewNG-Connection.h>
#include <iViewNG-Utility.h>

void iViewCallback (iViewTicket * const ticket) {}

int main (int argc, char ** args) {

    printf ("Starting...\n");

    CALL_API(iView_Init (IVIEWSDK_IVNG));
    CALL_API(iView_StartServer (NULL,NULL));
```

The function iView_Connect() is the first function which is able to return results asynchronously, e.g. indicating that a connection is lost. Therefor, you should create a user callback function and register it to the iViewNG API:

```
CALL_API(iView_SetCallback (iViewCallback));
```

The callback function is discussed later in this tutorial. Now you may create a ticket for the iView_-Connect() function and connect to the server using your favourite communication scheme:

```
iViewHost server;
memset (&server, 0, sizeof(iViewHost));
server.hostAddress.type = IVIEW_SERVERADRRESS_SHAREDMEMORY;

const unsigned int TIMEOUT_CONNECT = 1000; // milliseconds
const unsigned int RETRIES = 2; // number of attempts to retry to connect

iViewTicket * ticketConnect = NULL;

CALL_API(iView_CreateTicket (&ticketConnect));
CALL_API(iView_Connect (ticketConnect, &server, TIMEOUT_CONNECT,
      RETRIES, NULL ));
```

The connection is now ready to use.

Before using the API to control the eye tracker, please provide your license key to the iViewNG server, in this example, we use an empty string as license key:

```
iViewTicket * ticketAddLicense = NULL;

CALL_API(iView_CreateTicket (&ticketAddLicense));
CALL_API(iView_AddLicenseKey (ticketAddLicense, ""));
```

When done with your application, either just disconnect (and let the iViewNG server continue to operate) or terminate the server:

```
    printf ("Shutting down...\n");

    CALL_API(iView_ReleaseTicket (&ticketAddLicense));
    CALL_API(iView_ReleaseTicket (&ticketConnect));

    // terminate the iViewNG server and give it some time to shut down
    const unsigned int TIMEOUT_SHUTDOWN = 3000; // milliseconds
    CALL_API(iView_ShutdownServer (TIMEOUT_SHUTDOWN));

    // shut down the API
    CALL_API(iView_Shutdown ());

    return 0;
}
```

3.3 Step 3: Receiving Data Through the Callback

Goal of this example is that you understand how to handle asynchronously delivered data. You will use a function which returns asynchronously, implement a simple callback and understand how to extract the received data from the iViewTicket structure.

The source code of the example discussed in this section can be found in the SDK inside the folder *Tutorial-GetServerTime*.

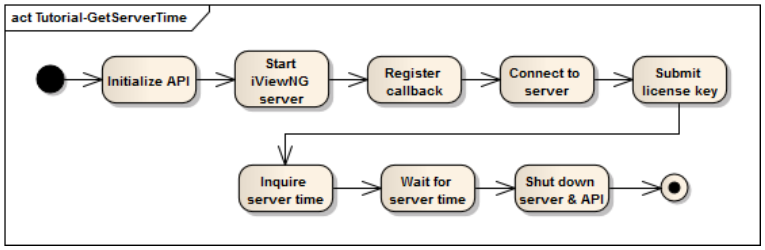


Figure 3.3: Caption

All asynchronously provided data are submitted using the user-defined callback function. The only parameter for this function is a pointer to an iViewTicket structure which points to the ticket which was used for the API command which caused the asynchronous output. For example, the API function iView_GetServerTime() creates asynchronous output indicating the server time.

Set up the server and connect to it as seen above, but do not forget to create a flag indicating if iView_-GetTime() already returned the result.

```
int main (int argc, char ** args) {

    printf ("Starting...\n");

    CALL_API(iView_Init (IVIEWSDK_IVNG));
    CALL_API(iView_StartServer (NULL,NULL));
    CALL_API(iView_SetCallback (iViewCallback));
```

```
    iViewHost server;
    memset (&server, 0, sizeof(iViewHost));
    server.hostAddress.type = IVIEW_SERVERADRRESS_SHAREDMEMORY;

    const unsigned int TIMEOUT_CONNECT = 1000; // milliseconds
    const unsigned int RETRIES = 2; // number of attempts to retry to connect

    CALL_API(iView_CreateTicket (&ticketConnect));
    CALL_API(iView_Connect (ticketConnect, &server,
      TIMEOUT_CONNECT, RETRIES, NULL ));

    CALL_API(iView_CreateTicket (&ticketAddLicense));
    CALL_API(iView_AddLicenseKey (ticketAddLicense, ""));
```

Call out for the server time:

```
printf ("Requesting current time...\n");

CALL_API(iView_CreateTicket (&ticketGetServerTime));
CALL_API(iView_GetServerTime (ticketGetServerTime));
```

and wait until the answer arrived or until the timeout limit is exceeded.

```
const unsigned int TIMEOUT_TICKET = 2000; // milliseconds
const unsigned int WAIT_PER_ITERATION = 50; // milliseconds
unsigned int waitingTime = 0; // the time we already waited for the result

do {
    iViewSleep (WAIT_PER_ITERATION); // let the process rest for some time
    waitingTime += WAIT_PER_ITERATION;
}
while (!receivedTime && waitingTime <= TIMEOUT_TICKET);
```

Clean up: release all tickets before they notify that the connection is lost and shut down the server.

```
    printf ("Shutting down...\n");

    CALL_API(iView_ReleaseTicket (&ticketGetServerTime));
    CALL_API(iView_ReleaseTicket (&ticketAddLicense));
    CALL_API(iView_ReleaseTicket (&ticketConnect));

    // terminate the iViewNG server and give it some time to shut down
    const unsigned int TIMEOUT_SHUTDOWN = 3000; // milliseconds
    CALL_API(iView_ShutdownServer (TIMEOUT_SHUTDOWN));

    // shut down the API
    CALL_API(iView_Shutdown ());

    return 0;
}
```

We define the sleep function so that it suits your operating system:

```
#ifdef WIN32
#include <windows.h>
#else
#include <unistd.h>
#endif
void iViewSleep (const unsigned int milliSeconds) {
#ifdef WIN32
    Sleep (milliSeconds);
```

```
#else
    usleep (1000 * milliSeconds);
#endif
}
```

The last required function is the callback function which compares the incoming ticket to all open tickets. This is possible since the tickets are globally accessable. In case that the ticket associated with the function iView_GetServerTime() is returned, we double-check for invalid pointers and retrieve the time the server is up and running:

```
void iViewCallback (iViewTicket * const ticket) {

    if (NULL == ticket) {
        printf ("ERROR: NULL ticket");
        return;
    }

    wchar_t * buffer = NULL;

    if (iView_iViewRcToString (&buffer, ticket->returnCode
      ))
        buffer = (wchar_t*) L"(invalid)";

    if (NULL != ticketConnect && ticket == ticketConnect && RC_NO_ERROR !=
      ticket->returnCode) {
        wprintf (L"ERROR: iViewConnect() returned returncode %u (%s)\n", ticket
      ->returnCode, buffer);
        free (buffer);
        return;
    }

    if (NULL != ticketAddLicense && ticket == ticketAddLicense && RC_NO_ERROR
      != ticket->returnCode) {
        wprintf (L"ERROR: iView_AddLicenseKey() returned returncode %u (%s)\n",
       ticket->returnCode, buffer);
        free (buffer);
        return;
    }

    if (NULL != ticketGetServerTime && ticket == ticketGetServerTime) {

        if (RC_NO_ERROR != ticket->returnCode) {
            wprintf (L"ERROR: iView_GetServerTime() returned returncode %u (%s)
      \n", ticket->returnCode, buffer);
            free (buffer);
            return;
        }

        // Avoid accessing NULL pointers and check if the result type matches
       the expected type!
        if (NULL == ticket->result || IVIEWRESULT_GET_SERVER_TIME !=
      ticket->result->type
                || NULL == ticket->result->data) {
            wprintf (L"ERROR: iView_GetServerTime() returned invalid result.\n"
      );
        }
        else {
            // If everything is allright, access the server time.
            uint64_t time = ((iViewTime *) ticket->result
      ->data)->time;
            printf ("Server time is: %4.1f milliseconds after process start.\n"
      , (time / 1000000.f));
            receivedTime = 1;
        }
    }
```

```
    return;
}
```

The code shown above is available as example *Example_GetServerTime*.

3.4 Step 4: Using the iViewNG Convenience Library

Together with the iViewNG API and the example source code, you received the source code of the iViewNG convenience library. This is a collection of functions and preprocessor macros for the C/C++ programming language which add functionality or makes it easier to handle the iViewNG API code.

For example, for simplicity, this tutorial does not need the API to operate asynchronously. In step 3 of this tutorial, you defined your own callback function and could .....

This library is open source and demonstrates the use of the iViewNG API. Use it for your convenience if you think the functions are useful. Please note that these functions are subject to future improvement and may change.

To use the iViewNG Convenience Library, just add the library to the list of libraries your compiler shall use, or take the

3.5 Step 5: Receiving Gaze Data

Goal of this example is to demonstrate how to receive data which is provided by the iViewNG as data stream. To keep the source code simple, we use the iViewNG Convenience Library.

The source code of the example discussed in this section can be found in the SDK inside the folder *Tutorial-DataAcquisition*.

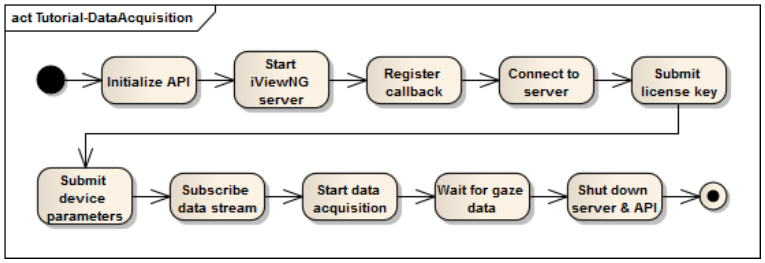


Figure 3.4: Caption

In many use cases, the client application needs to receive the gaze direction of the person using the eye tracking device. The gaze data is a result of the eye and gaze tracking algorithm which is part of the iViewNG server. The data structure which representing the gaze direction depends on the eye tracking device (e.g. mobile eye tracking device vs. fixed remote eye tracking device), but the workflow is shared. The user must

1. initialize the iViewNG server,
2. provide the required device parameters,
3. subscribe to the desired data stream,
4. start data acquisition.

This tutorial demonstrates a brief application which implements this workflow, utilizing the iViewNG Convenience Library.

First, we initialize the iViewNG server as we usually do:

```
#include <stdio.h>
#include <string.h> // for memset()

#include <iViewNG-Core.h>
#include <iViewNG-Connection.h>
#include <iViewNG-DataAcquisition.h>
#include <iViewNG-Device-ETG.h>
#include <iViewNG-Utility.h>
#include "iViewNG-Convenience.h"

unsigned int receivedGazeSamples = 0;
void iViewCallback (iViewTicket * const ticket);

int main (int argc, char ** args) {

    CALL_API(iView_Init (IVIEWSDK_IVNG));
    CALL_API(iView_StartServer (NULL,NULL));
    CALL_API(iView_SetCallback (iViewCallback));

    iViewHost server;
    memset (&server, 0, sizeof(iViewHost));
    server.hostAddress.type = IVIEW_SERVERADRRESS_SHAREDMEMORY;

    const unsigned int TIMEOUT_CONNECT = 1000; // milliseconds
    const unsigned int RETRIES = 2; // number of attempts to retry to connect

    iViewTicket * ticketConnect = NULL;

    CALL_API(iView_CreateTicket (&ticketConnect));
    CALL_API(iView_Connect (ticketConnect, &server,
      TIMEOUT_CONNECT, RETRIES, NULL ));

    iViewTicket * ticketAddLicense = NULL;

    CALL_API(iView_CreateTicket (&ticketAddLicense));
    CALL_API(iView_AddLicenseKey (ticketAddLicense, ""));
```

Second, we send the required parameters for an *iViewETG* device and block until the parameters are acknowledged. All parameters are neutral and use the ETG device just as it is initialized by default.

Please note the use of `iView_StartCatchingTickets()` and `iView_WaitForTicket()` which are no API functions, but open source convenience functions:

```
iViewDeviceParametersEtgCameraPlayback
      specificParameters;

specificParameters.outputDirectory = L".";
specificParameters.baseFilename = L"Tutorial-DataAcquisition";
specificParameters.operationMode = IVIEWTRACKINGMODE_CURRENT;
```

```
specificParameters.cameraResolutionScene =
      IVIEWRESOLUTION_CURRENT;
specificParameters.cameraResolutionEye =
      IVIEWRESOLUTION_CURRENT;
specificParameters.samplingRateSceneCam =
      IVIEWSAMPLINGRATE_CURRENT;
specificParameters.samplingRateEyeCam =
      IVIEWSAMPLINGRATE_CURRENT;
specificParameters.whiteBalanceProgram =
      IVIEWTRACKINGMODE_CURRENT;
specificParameters.audioState = IVIEWTRACKINGMODE_CURRENT;
specificParameters.overlaySpecList = NULL;

iViewDeviceParameters deviceParameters;

deviceParameters.deviceType = IVIEWDEVICE_ETG_CAMERAPLAYBACK;
deviceParameters.parameters = &specificParameters;

iViewTicket * ticketDeviceParameters = NULL;

CALL_API(iView_CreateTicket (&ticketDeviceParameters));
CALL_API(iView_StartCatchingTickets());
CALL_API(iView_SetDeviceParameters (
      ticketDeviceParameters, &deviceParameters));

if (false == iView_WaitForTicket (ticketDeviceParameters, 2000)) {
    printf ("Failed to set device parameters.\n");
    return -1;
}
```

Third, we subscribe to the gaze information data stream so that we receive data immediately after starting the data acquisition:

```
iViewStreamSubscription subscription;
memset (&subscription, 0, sizeof(iViewStreamSubscription
      ));
subscription.streamType = IVIEWDATASTREAM_GAZE_INFORMATION;

iViewTicket * ticketSubscription = NULL;

CALL_API(iView_CreateTicket(&ticketSubscription));
CALL_API(iView_StartCatchingTickets());
CALL_API(iView_SubscribeDataStream(ticketSubscription,
       &subscription));

if (false == iView_WaitForTicket (ticketSubscription, 2000)) {
    printf ("Failed to set subscribe data stream.\n");
    return -1;
}
```

Finally, we start data acquisition and again, we block until this is acknowledged:

```
iViewTicket * ticketStartAcquisition = NULL;

CALL_API(iView_CreateTicket(&ticketStartAcquisition));
CALL_API(iView_StartCatchingTickets());
CALL_API(iView_StartDataAcquisition(
      ticketStartAcquisition));

if (false == iView_WaitForTicket (ticketStartAcquisition, 2000)) {
    printf ("Failed to start data acquisition.\n");
    return -1;
}
```

At this point, our callback will receive gaze data. We explicitly wait until we receive 10 gaze samples or until the timeout strikes:

```
const unsigned int TIMEOUT_TICKET = 2000; // milliseconds
const unsigned int WAIT_PER_ITERATION = 50; // milliseconds
unsigned int waitingTime = 0; // the time we already waited for the result

do {
    iView_Sleep (WAIT_PER_ITERATION); // let the process rest for some time
    waitingTime += WAIT_PER_ITERATION;
}
while (9 > receivedGazeSamples && waitingTime <= TIMEOUT_TICKET);
```

Before terminating everything, we explicitly release all memory structures. This is not required since the application terminates anyway, but it's considered good style to release what you no longer need:

```
    iViewTicket * ticketUnsubscription = NULL;

    CALL_API(iView_CreateTicket(&ticketUnsubscription));
    CALL_API(iView_UnsubscribeDataStream(
      ticketUnsubscription, IVIEWDATASTREAM_GAZE_INFORMATION));

    iViewTicket * ticketStopAcquisition = NULL;

    CALL_API(iView_CreateTicket(&ticketStopAcquisition));
    CALL_API(iView_StopDataAcquisition(
      ticketStopAcquisition));

    CALL_API(iView_ReleaseTicket (&ticketConnect));
    CALL_API(iView_ReleaseTicket (&ticketAddLicense));
    CALL_API(iView_ReleaseTicket (&ticketDeviceParameters));
    CALL_API(iView_ReleaseTicket (&ticketStartAcquisition));
    CALL_API(iView_ReleaseTicket (&ticketSubscription));
    CALL_API(iView_ReleaseTicket (&ticketUnsubscription));
    CALL_API(iView_ReleaseTicket (&ticketStopAcquisition));

    // terminate the iViewNG server and give it some time to shut down
    const unsigned int TIMEOUT_SHUTDOWN = 3000; // milliseconds
    CALL_API(iView_ShutdownServer (TIMEOUT_SHUTDOWN));

    // shut down the API
    CALL_API(iView_Shutdown ());

    return 0;
}
```

When you wear the ETG device while executing the program shown above, you should see the gaze data printed to the console, for example:

```
Gaze Sample 1: time=3156ms, por=137.0/833.9
Gaze Sample 2: time=3189ms, por=567.3/792.1
Gaze Sample 3: time=3222ms, por=511.5/696.1
Gaze Sample 4: time=3255ms, por=517.7/677.6
Gaze Sample 5: time=3289ms, por=521.3/661.8
Gaze Sample 6: time=3322ms, por=525.5/652.7
Gaze Sample 7: time=3355ms, por=522.2/643.8
Gaze Sample 8: time=3389ms, por=531.2/641.4
Gaze Sample 9: time=3422ms, por=527.3/638.7
Gaze Sample 10: time=3455ms, por=527.9/634.7
```

3.6 Calibration

The iViewNG API provides a function set to perform a low-level calibration. That means, the API client is supposed to provide the coordinates of one or more points within the field of view the participant gazed at and the moment in time when this particular point was the point of regard.

To use the calibration,

```
example_nada(3); // Do nothing 3 times.
```

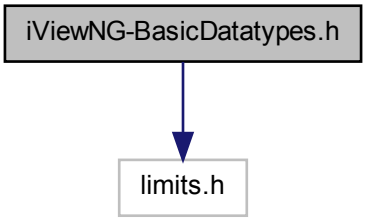
Chapter 4

Function Documentation

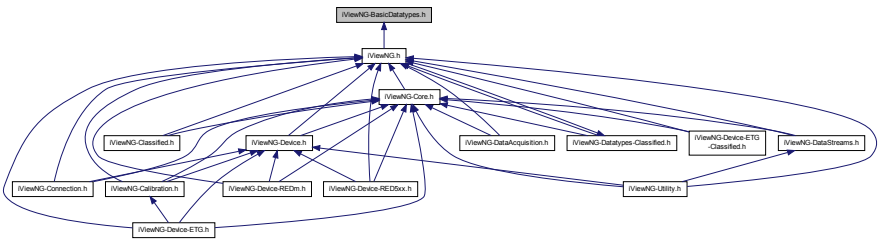
4.1 iViewNG-BasicDatatypes.h File Reference

`#include <limits.h>`

Include dependency graph for iViewNG-BasicDatatypes.h:



This graph shows which files directly or indirectly include this file:



Macro Definition Documentation

#define UINT_MAX 0xffffffff

Typedef Documentation

typedef signed char int8_t

8 bit datatypes

typedef unsigned char uint8_t

typedef short int16_t

16 bit datatypes

typedef unsigned short uint16_t

typedef int int32_t

32 bit datatypes

typedef unsigned uint32_t

typedef long long int64_t

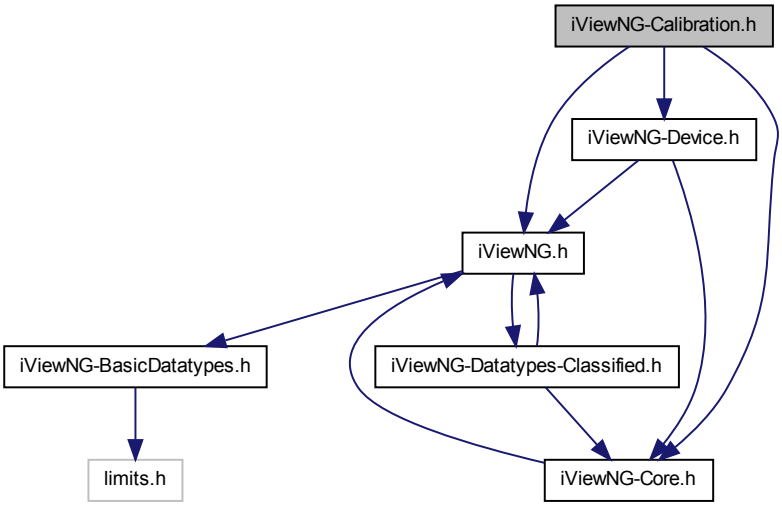
64 bit datatypes

typedef unsigned long long uint64_t

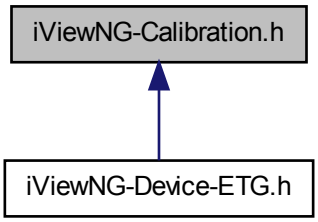
4.2 iViewNG-Calibration.h File Reference

```
#include <iViewNG.h>
#include <iViewNG-Core.h>
#include <iViewNG-Device.h>
```

Include dependency graph for iViewNG-Calibration.h:



This graph shows which files directly or indirectly include this file:



Data Structure Documentation

struct iViewCalibrationParameters

When calling iView_Calibration_Setup(), the user must provide information on the requested calibration type and parameters.

Data Fields

iView-Calibration-Type	type	The calibration method to use.

int32_t	applyAtFrame	Only meaningful for offline calibration: The scene frame index at which the calibration shall be applied. Use a value of -1 if the frame index is not known / not required and the server shall guess the proper value.
void *	parameters	Calibration scheme dependant data structure. Cast this according to the type, for details see the file iViewNG-Device-*.h.

struct iViewCalibrationData

For manual calibration of the gaze, the graphical client is expected to submit the 'real' gaze position in scene video coordinates. This means that at this frame, the subject was fixating the point (in 3D) described by the submitted coordinates (in 2D).

Data Fields

int32_t	sceneFrame-Number	The number of the scene frame (taken from SmiImage structure) at which the calibration was performed. If set to -1, the frame number is unknown and the server may make an educated guess on its own, e.g. use the frame number of the last acquired image. **See Also** iView_SubscribeDataStream Subscribing to IVIEWDATA-STREAM_SCENEIMAGES_WITH_GAZE or IVIEWDATAS-TREAM_SCENEIMAGES delivers iViewDataStreamScene-Image structures which contain the frame number to specify here.
int32_t	coordinateX	The horizontal position of the mouse click, in image pixel size, zero is leftmost column. The recording time when the mouse was pressed (timestamp relative to start of recording), as obtained with iView_GetRecordingTime()
int32_t	coordinateY	The vertical position of the mouse click, in image pixel size, zero is topmost row.

Typedef Documentation

typedef uint32_t iViewCalibrationType

Identifier for different calibration method (bit mask, may be combined to enumerate types).

Function Documentation

iViewRC iView_Calibration_Setup (iViewTicket * *ticket*, iViewCalibrationParameters const *const *setup*)

Initialize calibration procedure but do not start it.

Use this function to submit the calibration parameters to the iViewNG server. This function call specifies

- which calibration type to use (required)
- and when to apply the calibration (optional, only meaningful for offline calibration).

This function may be called at any time when a calibration procedure was not already started with iView_Calibration_Start() and not yet terminated by calling iView_Calibration_SendData() or iView_Calibration_Cancel().

The ticket progress will be set to 100 when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.
This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is NULL.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

Parameters

	inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead! •
in	*setup*	Pointer to parameter data structure.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if iView was not initialized by iView_Init()
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_INVALID_PARAMETER	if any parameter is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_Calibration_Start (iViewTicket * *ticket*)

Start the calibration procedure.

Call this function to indicate that a calibration procedure is in progress. This means, the function iView_Calibration_SendData() will be called soon after.

When iView_Calibration_Start() is called, the server might start to buffer calculated gaze samples. To save memory and CPU time, please finish a started calibration procedure by providing the proper amount of calibration data points (the number of required points depends on the calibration method submitted with iView_Calibration_Setup()) via iView_Calibration_SendData() or by canceling the procedure via iView_Calibration_Cancel().

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
The calibration parameters were supplied by iView_Calibration_Setup().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.
This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is NULL.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.

- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if iView was not initialized by iView_Init()
RC_INVALID_STATE	if iView_Calibration_Setup() was not called in advance
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_Calibration_SendData (iViewTicket * *ticket*, iViewCalibrationData const *const *data*)

Send calibration data for a single calibration point to the server.

The proper number of calibration data sets depends on the calibration type requested by iView_Calibration_Setup(). A 1-point-calibration requires one data set, a 3-point-calibration means the calibration is finished after calling iView_Calibration_SendData() three times etc. The client application must assert that either the proper number of calibration data structures is delivered by iView_Calibration_SendData() or the calibration procedure is canceled by iView_Calibration_Cancel().

Providing calibration data for a multi-point-calibration may have an immediate impact on the calculated gaze vector, even if the proper number of calibration data sets was not yet provided. The iView server may apply a calibration scheme which optimizes the gaze vector with the amount of calibration data structures given. Subsequent calls of iView_Calibration_SendData() will then further improve the calibration impact.

Sending invalid data may cancel a calibration procedure and acknowledge the failure to the API user.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
The calibration parameters were supplied by iView_Calibration_Setup().
The calibration procedure was started by calling iView_Calibration_Start().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.
This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is NULL.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

The successful delivery of calibration data will set the ticket associated with iView_Calibration_-Start() into the state IVIEWTICKETSTATUS_CLOSED and its return code to RC_NO_ERROR.

Parameters

	inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_-ReleaseTicket() instead!
in	*data*	Pointer to the parameter datastructure.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if iView was not initialized by iView_Init()
RC_CONNECTION_FAI-LED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_INVALID_PARAME-TER	if any parameter is invalid
RC_INVALID_STATE	if iView_Calibration_Setup() was not called in advance
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_Calibration_Cancel (iViewTicket * *ticket*)

Cancel an active calibration procedure.

The ticket progress will be set to 100 when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
The calibration parameters were supplied by iView_Calibration_Setup().
The calibration procedure was started by calling iView_Calibration_Start().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.
This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is NULL.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

The successful delivery of a cancel request will set the ticket associated with iView_Calibration_Start() into the state IVIEWTICKETSTATUS_CLOSED and its return code to RC_OPERATION_FAILED.

Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

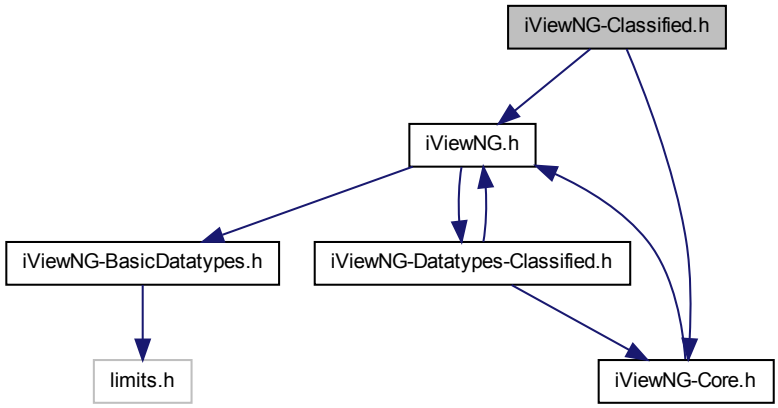
RC_NO_ERROR	on success
RC_NO_INIT	if iView was not initialized by iView_Init()

RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_INVALID_STATE	if iView_Calibration_Setup() was not called in advance
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

4.3 iViewNG-Classified.h File Reference

```
#include <iViewNG.h>
#include <iViewNG-Core.h>
```

Include dependency graph for iViewNG-Classified.h:



Function Documentation

iViewRC iView_LogString (unsigned int *level*, wchar_t const *const *message*)

Write the (timestamped) string message to (encrypted) logfile.

Note

SMI internal use only.

This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Precondition

The iView C API was initialized with iView_Init().

Parameters

in	*level*	specifies the level of the message: 1 = TRACE 2 = DEBUGGING 3 = VERBOSE 4 = INFO 5 = WARNING 6 = ERROR 7 = FATAL
in	*message*	the message itself

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_INVALID_PARAMETER	if any parameter is invalid
RC_UNSPECIFIED	on any other error

iViewRC iView_KillServer ( )

Kill all existing iViewNG server processes.

Note

Confidential. For SMI internal use only.

Warning

Do not use this function unless you are exactly knowing what you are doing. Use iView_Shutdown() instead!

This function does not produce asynchronous return values.

Return values

RC_NO_ERROR	on success
RC_INVALID_STATE	if an iViewNG server instance is already running (deprecated)
RC_OPERATION_FAILED	if the server could not be killed
RC_UNSPECIFIED	on any other error

iViewRC iView_CloseLogfile ( )

This function sends a message to the LoggerApp, that makes it close the current LogFile

Note

Confidential. For SMI internal use only.

This function does not produce asynchronous return values.

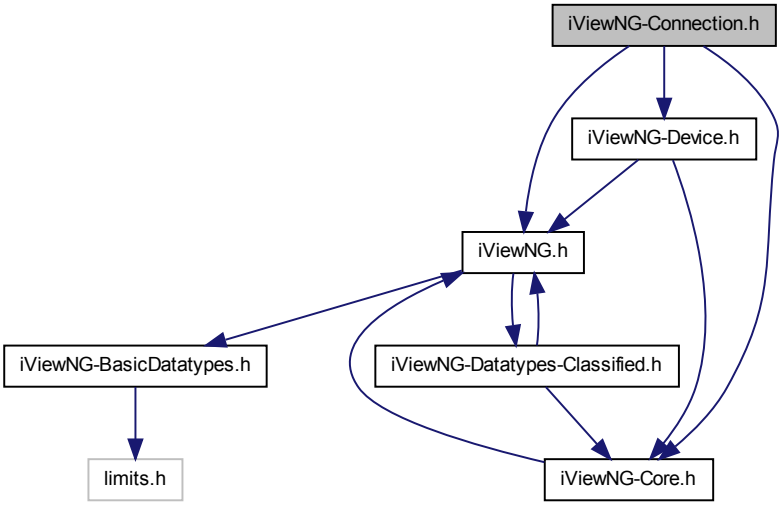
Return values

RC_NO_ERROR	on success
RC_OPERATION_FAILED	if a logger could not be created
RC_UNSPECIFIED	on any other error

4.4 iViewNG-Connection.h File Reference

```
#include <iViewNG.h>
#include <iViewNG-Core.h>
#include <iViewNG-Device.h>
```

Include dependency graph for iViewNG-Connection.h:



Data Structure Documentation

struct iViewHostAddress

Datastructure which represents the location of an iViewNG server instance.

Note

Do not create the iViewHostAddress structure manually. Generate with iView_CreateHostAddressIpv4() or iView_CreateHostAddressIpv6() or via iView_LocateServers().
Do not release the iViewHostAddress structure manually, use iView_ReleaseHostAddress() instead.

Data Fields

iViewServer-AddressType	type	State the type of the union: IPv4 or IPv6.
union iView-HostAddress	ipAddress	
unsigned short	udpPort	Base port for UDP communication. A value of 0 indicates that UDP communication is not available.
unsigned short	tcpPort	Base port for TCP communication. A value of 0 indicates that UDP communication is not available.

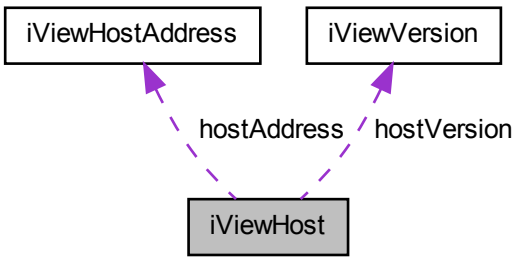
struct iViewHost

Data structure which represents one iViewNG server instance which is willing to provide access to one SMI device. If one iViewNG server is able to provide access to more than one SMI device, each device will be listed as one iViewHost.

Note

Generate via iView_LocateServers() to get all elements filled with proper values.
If not allocated manyally, do not release the iViewHost structure manually, use iView_ReleaseiView-Host() instead.

Collaboration diagram for iViewHost:



Data Fields

wchar_t	hostId	An optional string which the server may use to identify itself.
iViewHost-Address	hostAddress	The IP address & UDP/TCP port of a iViewNG server instance.
iViewVersion	hostVersion	Software version on host.
iViewDevice-Type	device	The SMI device which is provided/requested.

void *	capability	A set of provided/requested capabilities for this device. Device-dependant.

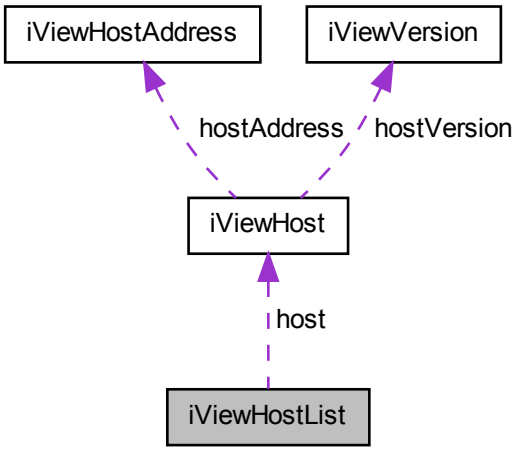
struct iViewHostListStruct

Data structure which contains a list of iViewNG server address structures.

Note

Do not create the iViewHostList structure manually. Generate via iView_LocateServers().
Do not release the iViewHostList structure manually, use iView_ReleaseiViewHostList() instead.

Collaboration diagram for iViewHostList:



Data Fields

iViewHost	host	Server host structure.
iViewHostList *	next	If more than one server is returned, this member points to the next structure. Defaults to NULL.

union iViewHostAddress.ipAddress

The server host address, either IPv4 or IPv6.

Data Fields

char	ipv4	The server host address, IPv4 version.
char	ipv6	The server host address, IPv6 version.

Typedef Documentation

typedef unsigned int iViewServerAddressType

Type to identify IP address types.

typedef unsigned int iViewLocationScope

Bit mask for defining restrictions for iView_LocateServers().

Function Documentation

iViewRC iView_Connect (iViewTicket *const *ticket*, iViewHost const *const *server*, const uint32_t *timeout*, const uint32_t *attempts*, iViewHost const *const *client*)

Connect to the specified iViewNG server.

This function blocks the caller until a connection was established or until the operation timed out. The ticket state remains IVIEWTICKETSTATUS_IN_PROGRESS until the connection was lost. When this happens, the ticket state will be set to IVIEWTICKETSTATUS_CLOSED, the progress is set to 100 and the return code is set to RC_CONNECTION_FAILED.

Before terminating, an established connection shall be closed using iView_Disconnect().

Precondition

The iView C API was initialized with iView_Init().

Postcondition

This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.state is IVIEWTICKETSTATUS_IN_PROGRESS as long as the connection is alive. When the connection was lost, this is delivered as error.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.

- iViewTicket.state is IVIEWTICKETSTATUS_IN_PROGRESS
- iViewTicket.progress is 100.

Parameters

	inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!
in	*server*	Specification of the server. Mandatory.
in	*timeout*	Do not try to connect to server for more than this time span [milliseconds]. It is recommended to wait not less than 1 seconds and retry 2 times.
in	*attempts*	Number of attempts to retry initiating a connection when the first attempt failed. Each attempt will approximately consume the time span defined by 'timeout'. When all attempts fail, the function will return indicating the failure. It is recommended to wait not less than 1 seconds and retry 2 times.
in	*client*	Specification of the client. Optional for legacy communication, not used for iVNG communication.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_INVALID_TICKET	if the ticket data structure is invalid
RC_INVALID_PARAMETER	if any of the parameters are invalid
RC_INVALID_STATE	if the server was already connected
RC_OPERATION_FAILED	if the connection attempt failed (e.g. because of timeout)
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_UNSPECIFIED	on any other error

iViewRC iView_Disconnect ( )

Cut the connection to the iViewNG server. Does not block pending events which may still arrive when this function completed.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

All pending tickets will be set to state IVIEWTICKETSTATUS_CLOSED, the return code is set to RC_NO_ERROR.

Return values

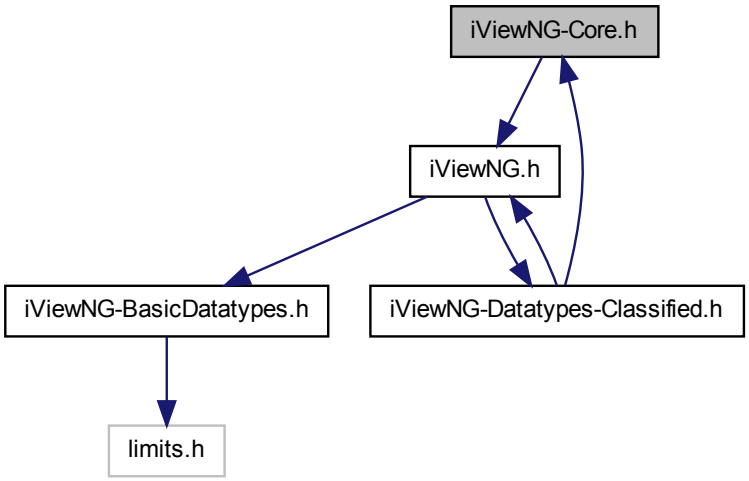
RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_UNSPECIFIED	on any other error

4.5 iViewNG-Convenience.h File Reference

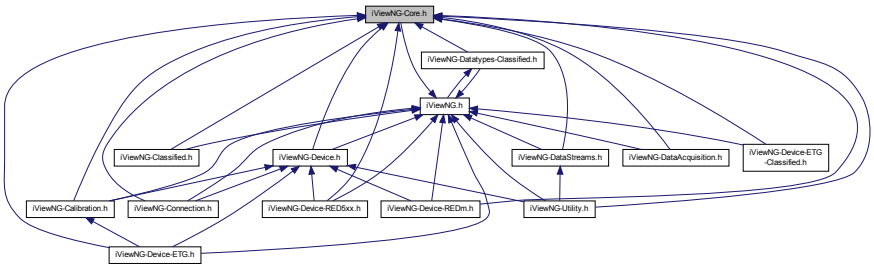
4.6 iViewNG-Core.h File Reference

```
#include <iViewNG.h>
```

Include dependency graph for iViewNG-Core.h:



This graph shows which files directly or indirectly include this file:



Data Structure Documentation

struct iViewVersion

Data structure to represent a software version.

Data Fields

uint32_t	major	The major version number.
uint32_t	minor	The minor version number.
uint32_t	patch	The patch version number.
uint32_t	build	The build number.

struct iViewTime

Data structure to represent a time scale.

Data Fields

uint64_t	time	Absolute time in nanoseconds.

struct iViewServerError

Data structure for error messages which also include a description.

Data Fields

iViewRC	errorCode	The server may generate a sophisticated error code to further declare what happened. Defaults to RC_UNSPECIFIED.
wchar_t *	message	A null-terminated human-readable message.

struct iViewVector2D

A structure for 2-dimensional position and direction vectors.

Data Fields

double	x	
double	y	

struct iViewVector3D

A structure for 3-dimensional position and direction vectors.

Data Fields

double	x	
double	y	
double	z	

struct iViewResultStruct

Every iViewTicket structure may contain a iViewResult data structure. This result structure is used to submit data back to the user application. To be able to determine the result data type, each iViewResult contains the data type of the included result. Use iViewResult.type to determine how to cast iViewResult.data.

Data Fields

iViewResultType	type	The type of the data structure data points to.
void *	data	The result data. Use the iViewResult.type to determine how to cast.
iViewResult *	next	If more than one result is submittet at once, this is a pointer to the next result structure. Defaults to NULL (no more data).

struct iViewOverlaySpec

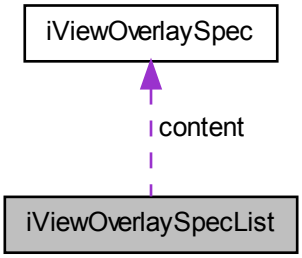
Data Fields

iViewOverlayType	type	
iViewOverlayPosition	positionHorizontal	

iViewOverlay-Position	positionVertical	
iViewOverlay-Option	option	

struct iViewOverlaySpecListStruct

Collaboration diagram for iViewOverlaySpecList:



Data Fields

iViewOverlay-Spec	content	
iViewOverlay-SpecList *	next	

struct iViewTicket

Datastructure for managing responses to API requests.

Never allocate an iViewTicket structure on your own, use iView_CreateTicket() instead.

Whenever an API function is called, the caller provides a pointer to a (not necessarily initialized) iView-Ticket structure. The iViewNG API writes a unique ticket number into the data structure and tags the request according to the processing state. The caller may at any time check if the request was already answered by examining the state indicator. While this flag is not set to 'IVIEWTICKETSTATUS_PROC-ESSED', the caller must not release the memory or modify its contents.

Upon completion, iViewNG may call the callback function with the iViewTicket as parameter. At this time, the requestPending flag is set to 'IVIEWTICKETSTATUS_PROCESSED'. If the called API function was

supposed to return data, the member 'result' is set to the result data structure.

The caller of the API functions is then responsible for explicitly indicating to release all data structures! iViewNG will not release neither iViewTicket structures nor the result data structure without the caller explicitly requesting to release the data! The user must call iView_ReleaseTicket() to let iViewNG release the iViewTicket structure and all contained structures.

Attention

Do not create the iViewTicket on your stack! Assert that the ticket memory is not explicitly or implicitly released or manipulated. This would corrupt your memory! If no longer required, call iView_release() to explicitly release the ticket data structure!

Data Fields

iViewTicket-Number	ticketNumber	The unique number of the request. Use this number to identify tickets. There will never be more than one ticket with this ticket number.
wchar_t	functionName	The name of the function call which was used for this ticket. There may be more than one ticket with this function name. Use this only for debugging purposes.
iViewTicket-State	state	The processing state of the ticket. Enumeration.
iViewTicket-Progress	progress	The progress of data acquisition. Not all command functions provide information about the progress, see the function documentation.
iViewRC	returnCode	A return code to indicate the result of the associated operation. Defaults to RC_NO_ERROR. A value different to RC_NO_ERROR indicates that 'result' points to an iViewServerError data structure if result != NULL. Otherwise, 'result' points to a iViewResult data structure.
iViewResult *	result	Pointer to the data returned by iView. Cast to iViewResult* if the return code indicates that the operation succeeded. Otherwise, cast to iViewServerError* to find an explanation.

Typedef Documentation

typedef uint32_t iViewRC

The return code for submitting the immediate result of a function call. This return code does not always indicate the result of the requested operation, but only the call to the function. Operation results are always returned by call-by-reference parameters like the iViewTicket data structure, synchronously and asynchronously.

typedef uint32_t iViewSdkType

Declaration of the SDK communication scheme.

typedef uint32_t iViewResultType

The type of the data structure which is addressed by iViewResult.data. To be able to determine the result data type, each iViewResult contains the data type of the included result. Use iViewResult.type to determine how to cast iViewResult.data.

typedef uint32_t iViewOverlayType

Definition of overlays to apply to images (bit mask, may be combined to list overlays).

typedef uint32_t iViewOverlayPosition

Define the position of an overlay (bit mask, may be combined to list overlays).

typedef uint32_t iViewOverlayOption

Additional parameters for the overlay. The meaning of the option depends on the overlay type.

Attention

Always initialize the structure with zero before using it!

typedef uint32_t iViewTicketNumber

Every iViewTicket contains a unique ticket number. Ticket numbers are managed by the iView API and must not be modified by the client.

typedef uint8_t iViewTicketState

Every iViewTicket contains a ticket state to indicate the current state of processing the request. The state describes the state of the API request itself, not the state of possibly collecting data from the iView server.

Note

Most API functions adjust the ticket state only after performing some initial checks, e.g. the system initialization. The purpose of the state is to indicate the request state based on asynchronous feedback from the server.

typedef uint8_t iViewTicketProgress

Every iViewTicket contains a progress indicator to provide the state of request progress with a percentage scale from 0 to 100 (request finished). Not all data requests provide information about the progress, see the function documentation.

typedef void(* IVIEW_CALLBACK)(iViewTicket *const)

Register the user callback function. The user callback function will be called asynchronously to provide any data to the user. A proper callback function is required for receiving data asynchronously.

It is possible to set the callback to NULL to indicate that no asynchronous results shall be delivered anymore and that all results shall be dismissed immediately.

This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in	*callback*	Pointer to the callback function. Use NULL to indicate that no results shall be delivered anymore and that all results shall be dismissed immediately.

Return values

RC_NO_ERROR	on success
RC_UNSPECIFIED	on any other error

Function Documentation

iViewRC iView_GetLibraryVersion (iViewVersion * *const*)

Retrieve the iViewNG library version numbers and write to provided structure.

This function may be called any time, even before calling iView_Init().

This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

inout]	version User-provided data structure to be filled.

Return values

RC_NO_ERROR	on success
RC_INVALID_PARAMETER	if the parameter is invalid (e.g. does not point to an initialized datastructure)

iViewRC iView_Init (iViewSdkType *sdkType*)

Initialize the iViewNG C API.

This function sets up the iView C API and puts it into the state where it may try to connect to the iViewNG server. It is safe but useless to try to initialize the API more than one time. If the API is no longer needed, call iView_Shutdown().

This function does not interact with the iViewNG server (if not connected to server). This function does not produce asynchronous return values.

Parameters

in	*sdkType*	defines the communication scheme (iViewNG vs. iViewX)

Return values

RC_NO_ERROR	on success
RC_INVALID_STATE	if the API was already initialized
RC_NO_INIT	if there is an initialization error
RC_INVALID_PARAMETER	if sdkType does not address a proper SDK type
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_Shutdown ( )

Deinitialize the API client and release the acquired memory. If still connected, the connection will be closed in advance.

This function does not interact with the iViewNG server (if not connected to server). This function does not produce asynchronous return values.

Precondition

The iView C API was initialized with iView_Init().

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_StartServer (wchar_t const *const *path*, wchar_t const *const *parameters*)

Start new iViewNG server with the provided command parameter set. The function blocks the caller until it is safe to assume that the server is up and ready.

This function does not produce asynchronous return values.

Parameters

in	*path*	reserved for future use (use NULL pointer)
in	*parameters*	reserved for future use (use NULL pointer)

Return values

RC_NO_ERROR	on success
RC_INVALID_STATE	if an iViewNG server instance is already running (deprecated)
RC_OPERATION_FAILED	if the server could not be started
RC_UNSPECIFIED	on any other error

iViewRC iView_ShutdownServer (const uint32_t *timeout*)

Terminate the iView server and release the acquired memory. This does not deinitialize the C API. Call iView_Shutdown() explicitly.

This function does not produce asynchronous return values.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Parameters

in	*timeout*	Do not wait for shutdown for more than this timeout [milliseconds]. It is recommended to wait for not less than 3 seconds.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if the ticket data structure is invalid / corrupted
RC_UNSPECIFIED	on any other error

iViewRC iView_SetCallback (DLLCALL *IVIEW_CALLBACK*)

IVIEW_CALLBACK iView_GetCallback ( )

Retrieve the address of the registered user callback function.

This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Returns

Pointer to the callback function registered with iView_SetCallback(), or NULL if no callback function was registered.

iViewRC iView_CreateTicket (iViewTicket **const *ticket*)

Dynamically generate a ticket data structure.

iView uses the ticket structure to identify a request, to indicate the state of processing the request and to provide the result data. Use iView_ReleaseTicket() to release the data structure and its contents. Do not release any contained data before calling iView_ReleaseTicket()!

This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

inout]	ticket Pointer to the iViewTicket data structure to initialize.

Return values

RC_NO_ERROR	on success
RC_INVALID_PARAMETER	if ticket is NULL or if the contents of ticket is not NULL
RC_OUT_OF_MEMORY	if the data structure could not be allocated
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_ReleaseTicket (iViewTicket **const *ticket*)

Recursively free the ticket data structure, including the result. All referred data allocated by iViewNG API is released. The ticket pointer will be set to NULL. Before releasing memory, the structure contents will be set to zero.

This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a request, to indicate the state of processing the request and to provide the result data. The user is responsible for not releasing this memory unless iView_ReleaseTicket() is called to release the result data structure.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_INVALID_TICKET	if ticket data structure is invalid
RC_OPERATION_FAILED	if ticket could not be released
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_ReleaseResult (iViewTicket *const *ticket*)

Recursively free the ticket result. All referred data allocated by iViewNG API is released.

This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a request, to indicate the state of processing the request and to provide the result data. The user is responsible for not releasing this memory unless iView_ReleaseTicket() is called to release the result data structure.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_INVALID_TICKET	if ticket data structure is invalid or contains no result
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_AddLicenseKey (iViewTicket *const *ticket*, char const *const *license*)

Provide user license key to iView NG. The license key is used to check the availability of functionality. It is possible to provide more than one license key, all license keys will be stored. If there is no connection to the server, the license(s) will be transferred implicitly when connecting to the server (

Precondition

The iView C API was initialized with iView_Init().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.
This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is NULL.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

Parameters

	inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!
`in`	*license*	The license string as provided to the user.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INVALID_PARAMETER	if any parameter is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_GetServerTime (iViewTicket *const *ticket*)

Retrieve the current time from the iViewNG server. Use this function for user-level synchronization of server and client application time scale.

The ticket progress will be set to 100 when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.
This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is a pointer to iViewResult,
  - iViewResult.type is IVIEWRESULT_GET_SERVER_TIME,
  - iViewResult.data is a pointer to iViewTime,
  - iViewResult.next is NULL since there will be no subsequent results.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized, see
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_GetLocalTime (iViewTicket *const *ticket*)

Retrieve the current time from the local host. Use this function for fetching the local time scale and for user-level synchronization of server and client application time scale.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Precondition

The iView C API was initialized with iView_Init().

Postcondition

The ticket state will be set to IVIEWTICKETSTATUS_CLOSED upon completion, the progress will be set to 100. The ticket will contain a result indicating the local time.

Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a request, to indicate the state of processing the request and to provide the result data. The user is responsible for not releasing this memory unless iView_ReleaseTicket() is called to release the result data structure.

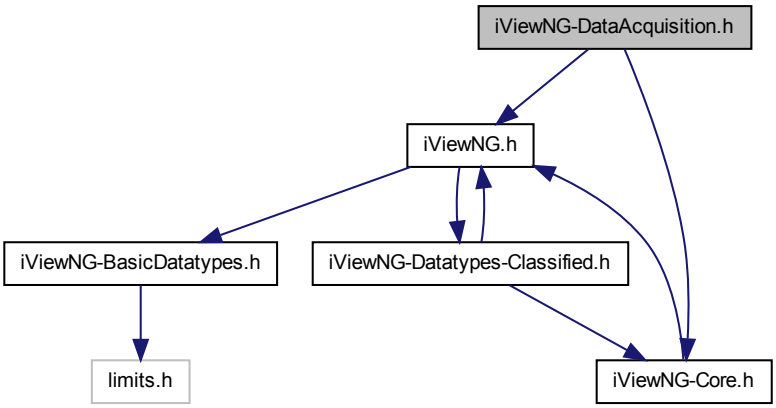
Return values

RC_NO_ERROR	
RC_NO_INIT	if the iViewNG C API was not initialized, see
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

4.7 iViewNG-DataAcquisition.h File Reference

```
#include <iViewNG.h>
#include <iViewNG-Core.h>
```

Include dependency graph for iViewNG-DataAcquisition.h:



Function Documentation

iViewRC iView_StartDataAcquisition (iViewTicket * *ticket*)

Trigger the start of data acquisition.

The ticket progress will be set to 100% when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
Device Parameters were set with iView_SetDeviceParameters().
A data stream provided by the sensor to start is acquired.

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.
This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- iViewTicket.resultType is IVIEWRESULT_DATA_ACQUISITION_START.
- On success:

  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.state is IVIEWTICKETSTATUS_IN_PROGRESS until the data acquisition is stopped actively or due to an error.
  - iViewTicket.progress is 50.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
  - iViewTicket.state is IVIEWTICKETSTATUS_CLOSED
  - iViewTicket.progress is 100.

Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INVALID_STATE	if a server-side precondition is not met
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_StopDataAcquisition (iViewTicket * *ticket*)

Trigger the end of data acquisition.

The ticket progress will be set to 100% when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
Device Parameters were set with iView_SetDeviceParameters().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.
This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.

- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED
  - iViewTicket.progress is 100.

Postcondition

On success, this function will close the ticket which called iView_StartDataAcquisition();

Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_Recording_Start (iViewTicket * *ticket*)

Ask the iViewNG server to start recording.

The ticket progress will be set to 100% when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
Device Parameters were set with iView_SetDeviceParameters().
Data acquisition is in progress.

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.
This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.state is IVIEWTICKETSTATUS_IN_PROGRESS until the recording is stopped actively or due to an error.
  - iViewTicket.progress is 50.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
  - iViewTicket.state is IVIEWTICKETSTATUS_CLOSED
  - iViewTicket.progress is 100.

Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_Recording_Stop (iViewTicket *)

Ask the iViewNG server to stop recording.

The ticket progress will be set to 100% when the request was processed by the server.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
Device Parameters were set with iView_SetDeviceParameters().
Recording is in progress.

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.
This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.

- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED
  - iViewTicket.progress is 100.

Postcondition

On success, this function will close the ticket which called iView_Recording_Start();

Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Return values

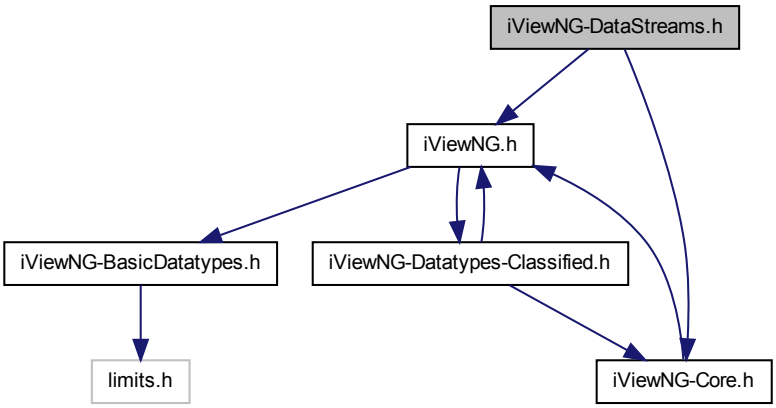
RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized

RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INVALID_STATE	if a server-side precondition is not met
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

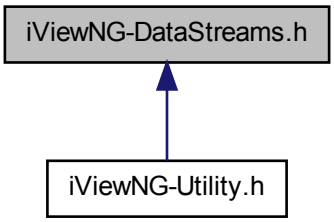
4.8 iViewNG-DataStreams.h File Reference

```
#include <iViewNG.h>
#include <iViewNG-Core.h>
```

Include dependency graph for iViewNG-DataStreams.h:



This graph shows which files directly or indirectly include this file:



Data Structure Documentation

struct iViewImage

constant for iViewImage.depth

The image structure which is used by iViewNG for providing non-compressed images. This structure is supposed to be compatible with the OpenCV image structure so that a iViewImage* can be cast to IplImage*. OpenCV is published under the following license:

IMPORTANT: READ BEFORE DOWNLOADING, COPYING, INSTALLING OR USING.

By downloading, copying, installing or using the software you agree to this license. If you do not agree to this license, do not download, install, copy or use the software.

```
                License Agreement
For Open Source Computer Vision Library
```

Copyright (C) 2000-2008, Intel Corporation, all rights reserved. Copyright (C) 2009, Willow Garage Inc., all rights reserved. Third party copyrights are property of their respective owners.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistribution's of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistribution's in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- The name of the copyright holders may not be used to endorse or promote products derived from this software without specific prior written permission.

This software is provided by the copyright holders and contributors "as is" and any express or implied warranties, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose are disclaimed. In no event shall the Intel Corporation or contributors be liable for any direct, indirect, incidental, special, exemplary, or consequential damages (including, but not limited to, procurement of substitute goods or services; loss of use, data, or profits; or business interruption) however caused and on any theory of liability, whether in contract, strict liability, or tort (including negligence or otherwise) arising in any way out of the use of this software, even if advised of the possibility of such damage.

Data Fields

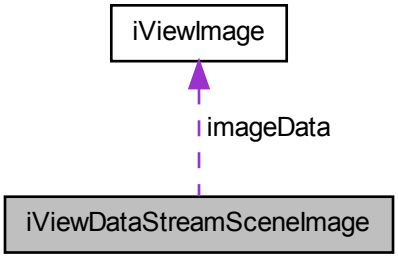
int32_t	dataSize	sizeof(iViewImage)
int32_t	id	version (=0)
int32_t	numberOf-Channels	number of colour channels
int32_t	alphaChannel	not used

int32_t	depth	pixel depth in bits: IVIEW_DEPTH_8U, IVIEW_DEPTH_8S, IVIEW_DEPTH_16S, IVIEW_DEPTH_32S, IVIEW_DEPTH_32F and IVIEW_DEPTH_64F are supported
int8_t	colorModel	not used
int8_t	channelSeq	not used
int32_t	dataOrder	0 - interleaved color channels, 1 - separate color channels
int32_t	origin	0 - top-left origin, 1 - bottom-left origin (Windows bitmaps style)
int32_t	align	not used
int32_t	width	image width in pixels
int32_t	height	image height in pixels
void *	roi	not used, always NULL
void *	maskROI	not used, always NULL
void *	imageId	not used, always NULL
void *	tileInfo	not used, always NULL
int32_t	imageSize	image data size in bytes, (==image->height*image->widthStep in case of interleaved data)
int8_t *	imageData	pointer to aligned image data
int32_t	widthStep	size of aligned image row in bytes
int32_t	BorderMode	not used
int32_t	BorderConst	not used
int8_t *	imageDataOrigin	pointer to very origin of image data (not necessarily aligned) - needed for correct deallocation

struct iViewDataStreamSceneImage

With iView_SubscribeDataStream(), the user may subscribe to data streams. If the user subscribes to the data stream IVIEWDATASTREAM_SCENEIMAGES_WITH_GAZE or IVIEWDATASTREAM_SCENEIMAGES, the user will receive iViewDataStream entities containing this data.

Collaboration diagram for iViewDataStreamSceneImage:



Data Fields

uint32_t	sceneFrame-Number	The number of the frame.
iViewImage *	imageData	The image structure.

struct iViewDataStreamH264SceneImage

With iView_SubscribeDataStream(), the user may subscribe to data streams. If the user subscribes to the data stream IVIEWDATASTREAM_SCENEIMAGES_H264, the user will receive iViewDataStream entities containing this data.

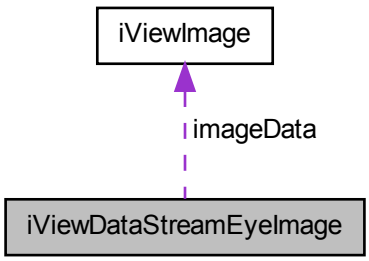
Data Fields

uint32_t	sceneFrame-Number	The number of the frame.
uint32_t	isKeyFrame	
uint32_t	dataSize	
void *	data	The raw H.264 data.

struct iViewDataStreamEyeImage

With iView_SubscribeDataStream(), the user may subscribe to data streams. If the user subscribes to the data stream IVIEWDATASTREAM_EYEIMAGES_LEFT or IVIEWDATASTREAM_EYEIMAGES_RIGHT, the user will receive iViewDataStream entities containing this data.

Collaboration diagram for iViewDataStreamEyeImage:



Data Fields

iViewEye-Identifier	eye	
uint32_t	eyeFrame-Number	Which eye is shown?
iViewImage *	imageData	The number of the frame.

struct iViewDataStreamTrackingStatus

This structure will be delivered when the user subscribes to IVIEWDATASTREAM_TRACKINGSTATUS and explains the current tracking quality.

Data Fields

int32_t	trackingQuality	An indicator for the integrated tracking quality of both eye. It indicates the confidence in the calculated gaze. A value in [0,100] where 0 means no confidence and 100 means highest confidence. If the trackingQuality is not calculated or otherwise invalid its value is set to -1.
int32_t	tracking-QualityLeft	An indicator for the tracking quality for the left eye. It indicates the confidence in the calculated gaze A value in [0,100] where 0 means no confidence and 100 means highest confidence. If the trackingQualityLeft is not calculated or otherwise invalid its value is set to -1.
int32_t	tracking-QualityRight	An indicator for the tracking quality for the left eye. It indicates the confidence in the calculated gaze A value in [0,100] where 0 means no confidence and 100 means highest confidence. If the trackingQualityRight is no calculated or otherwise invalid its value is set to -1.
int32_t	trackingRatio-Total	The tracking ratio (number of frames which resulted in gaze data / number of frames) in the range of [0..100]. The total tracking ratio describes the tracking ratio for the current recording. Please note that this value is only set when recording is active. A value of -1 indicates that there is no tracking ratio available at the moment.
int32_t	trackingRatio-Recent	The tracking ratio (number of frames which resulted in gaze data / number of frames) in the range of [0..100]. The recent tracking ratio describes the tracking ratio for a certain number of recent frames. Please note that this value is only set when recording is active. A value of -1 indicates that there is no tracking ratio available at the moment.
uint32_t	eyeFrameIndex	Frame index this trackability belongs to. A value of UINT_MAX indicates an invalid / unknown frame index.

double	relative-PositionY	this value represents the projection of the eye on the camera sensor the states can be: NAN location unknown 0 centered 1 upper maxiumum -1 lower maximum +INF outside, too high (if detected) -INF outside, too low (if detected) the shift between upper and lower boundary is a linear movement

struct iViewDataStreamSpecStruct

The basic layout of all stream specifiers.

Data Fields

iViewData-StreamSpec-Type	type	The type of this structure. Use this element for casting the whole structure.
iViewData-StreamSpec *	next	If there is another stream specifier, the address is contained here.

struct iViewDataStreamSpecSampleRate

Stream specifier to define a maximum framerate.

Data Fields

iViewData-StreamSpec-Type	type	Data inherited from iViewDataStreamSpec. Value must be IVIEWDATASTREAMSPEC_SAMPLE_RATE.
iViewData-StreamSpec *	next	Data inherited from iViewDataStreamSpec.
double	sampleRate	The desired receiving sample rate. Using this parameter it is possible to reduce the sample rate. Not all data streams understand this parameter (e.g. IVIEWDATASTREAM_SCENEIMAGES_H264). Use 0 to indicate no sample rate (no data will be sent at all). Use INFINITY to specify no sample rate limitation (as fast as server can send it). Use (0,INFINITY) to throttle the sample rate as specified. Using a negative value will have the same effect as using INFINITY (no sample rate limitation).

struct iViewDataStreamSpecGazeFilter

Stream specifier to apply a gaze filter. Value must be IVIEWDATASTREAMSPEC_GAZE_FILTER.

Data Fields

iViewDataStreamSpecType	type	Data inherited from iViewDataStreamSpec.
iViewDataStreamSpec *	next	Data inherited from iViewDataStreamSpec.
int32_t	gazeFilterType	Apply a gaze filter before overlaying gaze cursor. 0: no filter 1: IIR filter 2: FIR filter

struct iViewDataStreamSpecQualityOfService

Stream specifier to specify the required quality of service for remote communication.

Data Fields

iViewDataStreamSpecType	type	Data inherited from iViewDataStreamSpec. Value must be IVIEWDATASTREAMSPEC_QUALITY_OF_SERVICE.
iViewDataStreamSpec *	next	Data inherited from iViewDataStreamSpec.
iViewQualityOfService	quality	Specify the required quality of service for remote communication.

struct iViewStreamSubscription

Datastructure to specify a data stream subscription. The contents may vary according to the device type.

Note

If it is some day required to add stuff for other devices, change this to a union, depending on the device.

Data Fields

iViewDataStreamType	streamType	The type of the datastream.

iViewData-StreamSpec *	streamSpec	Start of a linked list of (possibly device-dependent) sream specifiers. Defaults to NULL. Use iViewDataStreamSpec... data structures and link them together.

struct iViewDataStreamEndOfStream

With iView_SubscribeDataStream(), the user may subscribe to data streams. In return, the user will receive iViewResult structures containing one or more iViewDataStream entities. When the end of a data stream is reached, e.g. because the data stream was provided by a file player or a sensor terminated submission of data on its own, a iViewDataStream structure is delivered which indicates that the end of the stream was reached. The data pointer of this structure then points to a iViewDataStreamEndOf-Stream structure.

Data Fields

uint32_t	numberOf-Samples	The number of samples which have been created since the last call of startDataAcquisition(), excluding the EndOfStream data stream entity. Use this information to assert that no samples were lost in transmission.

struct iViewDataStream

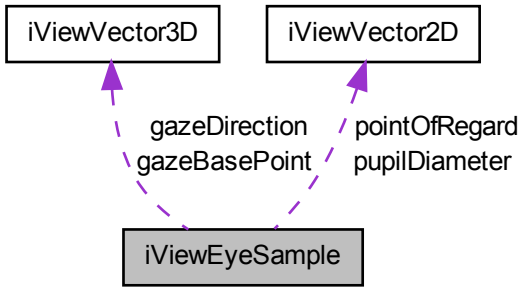
With iView_SubscribeDataStream(), the user may subscribe to data streams. In return, the user will receive iViewResult structures containing one or more iViewDataStream entities.

Data Fields

iViewData-StreamType	type	
uint32_t	index	A counter which contains a running number. The index enumerates the number of valid stream entities. With each provided data entity, this number is increased by 1, starting at 0 only when the application starts.
iViewData-StreamState	lastStream-Entity	A flag to indicate whether this is the last stream entity (lastStream-Entity := IVIEWDATASTREAM_END) or not and if more stream entities will be provided subsequently (lastStreamEntity := IVIEW-DATASTREAM_NO_END). Set to IVIEWDATASTREAM_END_-UNKNOWN if unknown. If set to IVIEWDATASTREAM_END, the data pointer refers to a iViewDataStreamEndOfStream structure.
void *	data	Pointer to the data (may be NULL, e.g. if lastStreamEntity == 2).

struct iViewEyeSample

Collaboration diagram for iViewEyeSample:

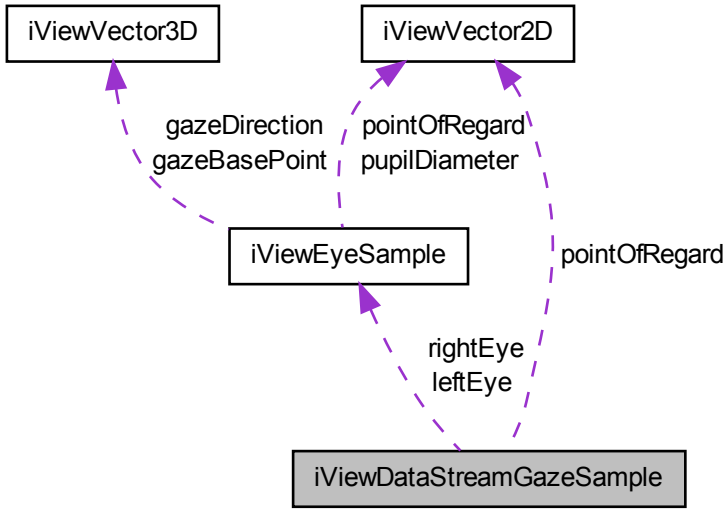


Data Fields

iViewVector2D	pupilDiameter	Diameter of the left Pupil [eye camera pixels]. Values of NAN means that the diameter was not calculated.
double	pupilRadius	Pupil radius [mm]. A value of NAN means that the radius was not calculated.
double	pupil-Confidence	Confidence of the left and right pupil A value of NAN means that the confidence was not calculated.
iViewVector2D	pointOfRegard	The point of regard, in screen coordinates. A value of NAN means that the coordinate is invalid and the POR could not be calculated.
iViewVector3D	gazeBasePoint	...
iViewVector3D	gazeDirection	3D Gaze Direction vector based in 3D gaze base point

struct iViewDataStreamGazeSample

Collaboration diagram for iViewDataStreamGazeSample:



Data Fields

uint32_t	sceneFrame-Number	The number of the scene frame this POR corresponds to.
uint32_t	eyeFrame-Number	The number of the eye frame this POR corresponds to.
uint64_t	timestamp	Timestamp of the GazeSample in nanoseconds.
iViewVector2D	pointOfRegard	The point of regard, in screen coordinates. A value of NAN means that the coordinate is invalid and the POR could not be calculated.
iViewEye-Sample	leftEye	The left eye data.
iViewEye-Sample	rightEye	The right eye data.

Typedef Documentation

typedef uint32_t iViewEyeIdentifier

Identifier to determine the eye.

typedef uint32_t iViewDataStreamType

If the user wants to asynchronously receive data, the user must subscribe to the data stream in advance. The iViewDataStreamType type is used to specify the kind of data the user wants to receive.

typedef uint32_t iViewDataStreamSpecType

typedef uint32_t iViewQualityOfService

Identifier of different comunication qualities.

typedef uint32_t iViewDataStreamState

Identifier to differ between alive streams (which will continue to send data) and terminated streams (which will not submit any further data).

Function Documentation

iViewRC iView_SubscribeDataStream (iViewTicket * *ticket,* iViewStreamSubscription const *const *subscription*)

Subscribe a datastream. As consequence, the API will receive the subscribed data asynchronously and eventually call the callback function to submit the data to the API user.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKET-STATUS_ACCEPTED.
This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

Parameters

	inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_-ReleaseTicket() instead!
in	*subscription*	type and constraints for the subscription, may be released immediately after the function returns

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INVALID_PARAMETER	if any parameter is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

iViewRC iView_UnsubscribeDataStream (iViewTicket * *ticket*, const iViewDataStreamType *streamType*)

Unsubscribe a datastream of the specified type. As consequence, the API will no longer receive the subscribed data asynchronously.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().
The specified stream was subscribed.

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.
This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

Postcondition

On success, this function will close the ticket which called iView_SubscribeDataStream();

Parameters

	inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!
in	*streamType*	the desired stream type

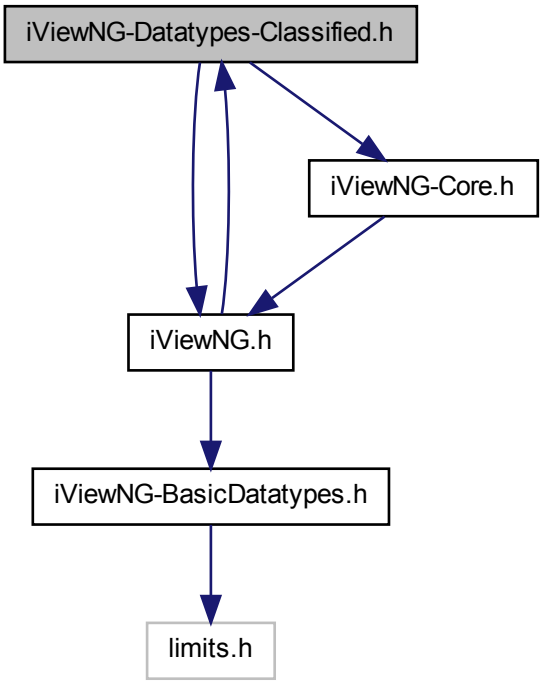
Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INVALID_PARAMETER	if any parameter is invalid
RC_INTERNAL_ERROR	if an internal error occurred
RC_UNSPECIFIED	on any other error

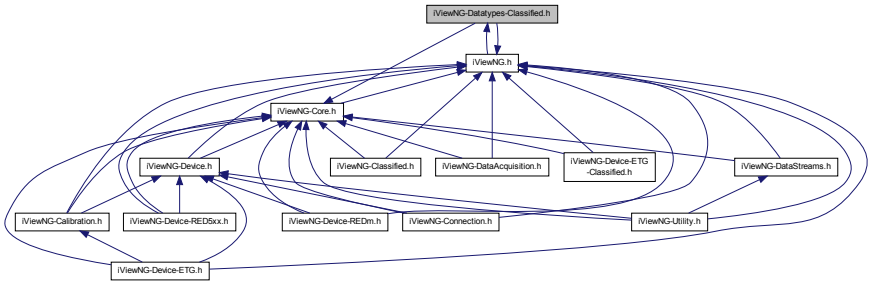
4.9 iViewNG-Datatypes-Classified.h File Reference

```
#include <iViewNG.h>
#include <iViewNG-Core.h>
```

Include dependency graph for iViewNG-Datatypes-Classified.h:



This graph shows which files directly or indirectly include this file:



Data Structure Documentation

struct iViewIdfPlaybackData

The -classified- C API datatypes ate used internally only e.g. by iViewETG-Client, ExperimentCenter, BeGaze.

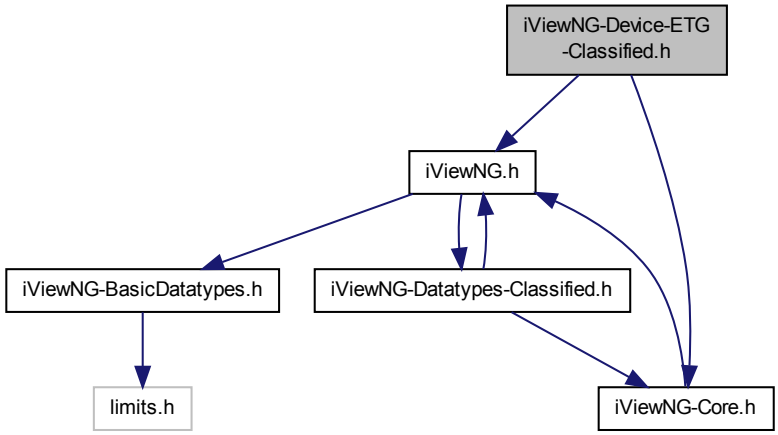
Data Fields

wchar_t const *	pathIdfFile	The path of the Idf file. The server process must be able to access this path. It must not end with a path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
double	Playback-Frequency	The frame rate to use for playing back the idf File. A value of Infinity means 'as fast as possible'. A value of NAN means 'no change'.

4.10 iViewNG-Device-ETG-Classified.h File Reference

```
#include <iViewNG.h>
#include <iViewNG-Core.h>
```

Include dependency graph for iViewNG-Device-ETG-Classified.h:



Data Structure Documentation

struct iViewDeviceParametersEtgVideoPlayback

Used in iViewDeviceParameters if iViewDeviceType is IVIEWDEVICE_ETG_VIDEOPLAYBACK. Includes the parameters for using realtime playback of an ETG device (aka Mii, Smiggles, GazeWear, HED5). If all parameters are set to zero, the parameters will not be changed but remain in their current state.

Note

Confidential

Attention

Always initialize the structure with zero before using it!

Data Fields

wchar_t const *	outputDirectory	The directory for storing recorded files, NOT ending with the path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
wchar_t const *	baseFilename	The filename to append to outputDirectory, NOT including any suffix, NOT including any path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
wchar_t const *	pathEyeVideo-Files	The path of the eye video files. The server process must be able to access this path. It must not end with a path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.

wchar_t const *	leftEyeVideoFile	The filename of the left eye video. Must not contain the path. A value of NULL means 'no change' and orders the server to not change the last submitted filename.
wchar_t const *	rightEyeVideoFile	The filename of the right eye video. Must not contain the path. A value of NULL means 'no change' and orders the server to not change the last submitted filename.
wchar_t const *	pathSceneVideoFiles	The path of the scene video file. The server process must be able to access this path. It must not end with a path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
wchar_t const *	sceneVideoFile	The filename of the scene video. Must not contain the path. A value of NULL means 'no change' and orders the server to not change the last submitted filename.
wchar_t const *	pathEyeCameraConfigFiles	The path of the camera config files. The server process must be able to access this path. It must not end with a path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
wchar_t const *	leftEyeCameraConfigFile	The filename of the eye camera configuration. Must not contain the path. A value of NULL means 'no change' and orders the server to not change the last submitted filename.
wchar_t const *	rightEyeCameraConfigFile	The filename of the eye camera configuration. Must not contain the path. A value of NULL means 'no change' and orders the server to not change the last submitted filename.
double	framesPerSecondLeftEye	The frame rate to use for playing back the left eye video. A value of Infinity means 'as fast as possible' (not yet implemented). A value of NAN means 'no change'. **Note** Currently, left and right eye may only use the same frame rate.
double	framesPerSecondRightEye	The frame rate to use for playing back the right eye video. A value of Infinity means 'as fast as possible' (not yet implemented). A value of NAN means 'no change'. **Note** Currently, left and right eye may only use the same frame rate.
double	framesPerSecondScene	The frame rate to use for playing back the eye video. A value of Infinity means 'as fast as possible' (not yet implemented). A value of NAN means 'no change'.
int8_t	loopPlayback	Indicator whether the videos should be played in a loop (1) or not (-1). A value of 0 indicates not to change the last submitted value.

struct iViewDeviceParametersEtgIdfPlayback

Used in iViewDeviceParameters if iViewDeviceType is IVIEWDEVICE_ETG_IDFPLAYBACK. Includes the parameters for using realtime playback of an ETG device (aka Mii, Smiggles, GazeWear, HED5). If all parameters are set to zero, the parameters will not be changed but remain in their current state.

Note

Confidential

Attention

Always initialize the structure with zero before using it!

Data Fields

wchar_t const *	pathIdfFile	The path of the Idf file. The server process must be able to access this path. It must not end with a path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
wchar_t const *	eyeDataFile	The filename of the IDF. Must not contain the path. A value of NULL means 'no change' and orders the server to not change the last submitted filename.
double	playbackFrequency	The frame rate to use for playing back the idf File. A value of Infinity means 'as fast as possible'. A value of NAN means 'no change'.

struct iViewDeviceParametersEtgIdfRecalibrationPlayback

Used in iViewDeviceParameters if iViewDeviceType is DEVICE_PARAMETERS_ETG_IDF_PLAYBACK_RECALIBRATION. Includes the parameters for using realtime playback of an ETG device (aka Mii, Smiggles, GazeWear, HED5) with additional recalibration of raw gaze samples. If all parameters are set to zero, the parameters will not be changed but remain in their current state.

Note

Confidential

Attention

Always initialize the structure with zero before using it!

Data Fields

wchar_t const *	pathIdfFile	The path of the Idf file. The server process must be able to access this path. It must not end with a path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
wchar_t const *	eyeDataFile	The filename of the IDF. Must not contain the path. A value of NULL means 'no change' and orders the server to not change the last submitted filename.
double	playback-Frequency	The frame rate to use for playing back the idf File. A value of Infinity means 'as fast as possible'. A value of NAN means 'no change'.

struct iViewDeviceParametersEtgRuVideoPlayback

Used in iViewDeviceParameters if iViewDeviceType is IVIEWDEVICE_ETG_RU_VIDEOPLAYBACK. Includes the parameters for offline processing of a recording done by an ETG recording unit. If all parameters are set to zero, the parameters will not be changed but remain in their current state.

Note that this device type will let server start a recording and playback automatically. No event subscriptions are necessary to start the acqusition/recording. BeGaze usually does subscribe gaze samples, as it wants to monitor the progress.

Attention

Always initialize the structure with zero before using it!

Data Fields

wchar_t const *	pathUnified-ReaderFile	The path to the unified file.
wchar_t const *	unifiedReader-File	The unified file (the recording of a recording unit) to be processed offline.
wchar_t const *	outputDirectory	The directory for storing recorded files, NOT ending with the path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
wchar_t const *	baseFilename	The filename to append to outputDirectory, NOT including any suffix, NOT including any path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.

struct iViewGazeCorrectionParameters

With calling iView_SetGazeCorrectionData(), the user must provide information on calibration type and parameters.

Data Fields

wchar_t const *	xmlStream	Correction scheme dependant XML string, as loaded from gaze correction data file.
int32_t	applyAtFrame	Only meaningful for offline calibration: The scene frame index at which the calibration shall be applied. This overrides the frame index included in the XML string! Use a value of -1 to indicate to not override the XML element and use the default.

Function Documentation

iViewRC iView_SetParticipantData (iViewTicket * *const*, wchar_t const * *const*)

The -classified- C API function is used internally only e.g. by iViewETG-Client, ExperimentCenter, BeGaze.

iViewRC iView_GetParticipantData (iViewTicket * *const*)

The -classified- C API function is used internally only e.g. by iViewETG-Client, ExperimentCenter, BeGaze.

iViewRC iView_SetExperimentData (iViewTicket * *const*, wchar_t const * *const*)

The -classified- C API function is used internally only e.g. by iViewETG-Client, ExperimentCenter, BeGaze.

iViewRC iView_GetExperimentData (iViewTicket * *const*)

Download experiment data from the ETG device. The loaded data will be returned and written into the data repository.

Postcondition

This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is a pointer to ...,

  - iViewResult.type is ...,
  - iViewResult.data is a pointer to ...,
  - iViewResult.next is NULL since there will be no subsequent results.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

iViewRC iView_SetGazeCorrectionData (iViewTicket * *const*, iViewGazeCorrectionParameters const * *const*)

Provide gaze correction data as it would have been created by an online calibration. This is only useful for offline data processing. This is only useful for offline processing.

Postcondition

This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success:
  - iViewTicket.returnCode is RC_NO_ERROR,
  - iViewTicket.result is NULL,
  - iViewResult.next is NULL since there will be no subsequent results.
- On failure:
  - iViewTicket.returnCode indicates the error code,
  - iViewTicket.result is NULL or a pointer to iViewServerError.
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

Note

SMI internal use only.

iViewRC iView_GetGazeCorrectionData (iViewTicket * *const*)

Trigger the download of gaze correction data from the ETG device. The stored gaze correction data will be written to the data repository.

Note

SMI internal use only.

Postcondition

This function returns asynchronically and provides the result of the operation to the user callback function. Use the ticket to associate the operation request and the return of the result. On return, the ticket's elements are set:

- On success: ...
- On failure: ...
- iViewTicket.progress is 100.
- iViewTicket.state is IVIEWTICKETSTATUS_CLOSED since there will be no subsequent call with this ticket. The ticket should be released.

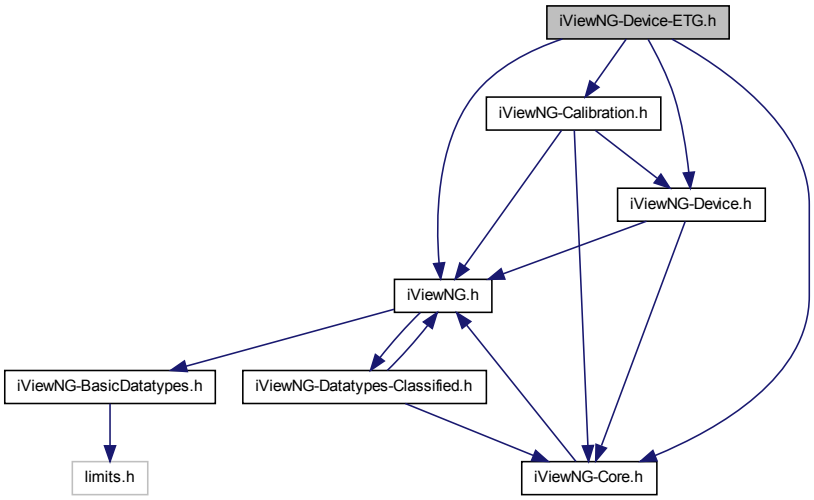
Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a command request when delivering results asynchronously. Use iView_CreateTicket() to initialize a ticket data structure. The ticket must not be released manually! Use iView_ReleaseTicket() instead!

4.11 iViewNG-Device-ETG.h File Reference

```
#include <iViewNG.h>
#include <iViewNG-Core.h>
#include <iViewNG-Device.h>
#include <iViewNG-Calibration.h>
```

Include dependency graph for iViewNG-Device-ETG.h:



Data Structure Documentation

struct iViewDeviceCapabilityEtg

Data structure to describe a capability set of an ETG device.

Data Fields

iViewDevice-Type	playback-Modes	
iViewTracking-Mode	acquisition-Modes	
iView-Calibration-Type	calibration-Types	
iViewCamera-Resolution	sceneImage-Resolutions	
iView-SamplingRate	sceneImage-SampleRates	
iViewCamera-Resolution	eyeImage-Resolutions	

iView-SamplingRate	eyeImage-SampleRates	

struct iViewDeviceVersionEtg

Data Fields

int	software-VersionScene-Cam	Scene camera software version, format YYYYMMDD or -1 if unknown.
int	firmware-VersionEye-Cam	Scene camera firmware version number or -1 if unknown. Eye camera firmware version number or -1 if unknown.

struct iViewDataStreamEtgServerStatus

Structure to represent the recording device for an ETG system.

Data Fields

int32_t	battery-Lifetime-Minutes	The remaining battery lifetime in number of minutes. A value of INT_MAX indicates that the recording device does not operate on battery but on AC power. A value of -1 indicates that the battery state could not be determined.
int32_t	battery-Lifetime-Percent	The remaining battery lifetime in percent. A value of -1 indicates that the battery state could not be determined.
uint32_t	freeStorage-Minutes	The currently free storage space on recording unit data tank, given in recording minutes. This describes the device which contains the data repository. A value of UINT_MAX indicates that the capacity exceeds UINT_MAX-1 minutes by an unknown size.
int32_t	sceneCamera-Available	The status of the scene camera. If 1, the camera is operational and may be started. If 0, the camera is not (yet) ready.
int32_t	eyeCamera-Available	The status of the eye camera. If 1, the camera is operational and may be started. If 0, the camera is not (yet) ready.
double	systemWide-CpuUsage	The current system wide cpu usage over all cpu cores (the load of the computer). In [0,100]. Can be -1 on error.
uint32_t	recordingTime	The length of the recording in progress or (if recording is not active) the length of the last acquired recording [in milliseconds].

struct iViewDeviceParametersEtgCameraPlayback

Used in iViewDeviceParameters if iViewDeviceType is IVIEWDEVICE_ETG_CAMERAPLAYBACK. Includes the parameters for using realtime playback of an ETG device (aka Mii, Smiggles, GazeWear, HED5). If all parameters are set to zero, the parameters will not be changed but remain in their current state.

Attention

Always initialize the structure with zero before using it!

Data Fields

wchar_t const *	outputDirectory	The directory for storing recorded files, NOT ending with the path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
wchar_t const *	baseFilename	The filename to append to outputDirectory, NOT including any suffix, NOT including any path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
iViewTracking-Mode	operationMode	The operation mode(s) of the eye tracker. A value of IVIEWTRACKINGMODE_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iViewCamera-Resolution	camera-Resolution-Scene	The resolution of the submitted scene images [pixels] A value of IVIEWRESOLUTION_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iViewCamera-Resolution	camera-ResolutionEye	The resolution of the submitted eye images [pixels] A value of IVIEWRESOLUTION_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iView-SamplingRate	samplingRate-SceneCam	Average scene camera frame rate [frames per second] A value of IVIEWSAMPLINGRATE_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iView-SamplingRate	samplingRate-EyeCam	Average scene camera frame rate [frames per second] A value of IVIEWSAMPLINGRATE_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iViewWhite-Balance-Program	whiteBalance-Program	Scene camera autowhitebalance program. A value of IVIEWTRACKINGMODE_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iViewAudio-ActivationState	audioState	Audio activation state. A value of IVIEWTRACKINGMODE_CURRENT means 'no change' and orders the server to not change the last submitted setting.

iViewOverlay-SpecList *	overlaySpec-List	**Note** not yet supported

struct iViewCalibrationParametersEtg

Data structure including all required parameters for calibrating an ETG device.

Data Fields

double	calibration-Distance	The calibration distance [m] which will be used. A value of 0 indicates that the calibration distance is unknown / not used.

struct iViewDeviceParametersEtgRuRealtime

Used in iViewDeviceParameters if iViewDeviceType is IVIEWDEVICE_ETG_RU_REALTIME. Includes the parameters for using recording unit for an ETG device. If all parameters are set to zero, the parameters will not be changed but remain in their current state.

Attention

Always initialize the structure with zero before using it!

Data Fields

wchar_t const *	outputDirectory	The directory for storing recorded files, NOT ending with the path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
wchar_t const *	baseFilename	The filename to append to outputDirectory, NOT including any suffix, NOT including any path separator. A value of NULL means 'no change' and orders the server to not change the last submitted path.
iViewTracking-Mode	operationMode	The operation mode(s) of the eye tracker. A value of IVIEWTRACKINGMODE_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iViewCamera-Resolution	camera-Resolution-Scene	The resolution of the submitted scene images [pixels] A value of IVIEWRESOLUTION_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iViewCamera-Resolution	camera-ResolutionEye	The resolution of the submitted eye images [pixels] A value of IVIEWRESOLUTION_CURRENT means 'no change' and orders the server to not change the last submitted setting.

iViewSamplingRate	samplingRateSceneCam	Average scene camera frame rate [frames per second] A value of IVIEWSAMPLINGRATE_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iViewSamplingRate	samplingRateEyeCam	Average scene camera frame rate [frames per second] A value of IVIEWSAMPLINGRATE_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iViewWhiteBalanceProgram	whiteBalanceProgram	Scene camera autowhitebalance program. A value of IVIEWWHITEBALANCE_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iViewAudioActivationState	audioState	Audio activation state. A value of IVIEWAUDIOSTATE_CURRENT means 'no change' and orders the server to not change the last submitted setting.
iViewOverlaySpecList *	overlaySpecList	**Note** not yet supported A value of IVIEWTRACKINGMODE_CURRENT means 'no change' and orders the server to not change the last submitted setting.

Variable Documentation

const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_ETG_AUTO = 0x01

Do not change the current setting.

Automatically adjust white balance setting.

const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_ETG_DAYLIGHT = 0x02

For using outside, clear days.

const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_ETG_BASIC = 0x04

For using outside, cloudy or in shade.

const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_ETG_TUNGSTEN = 0x08

For using in tungsten lightning.

const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_ETG_FLUORESCENT = 0x10

For using in white fluorescent lightning.

const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_ETG_FLUORESCENT_H = 0x20

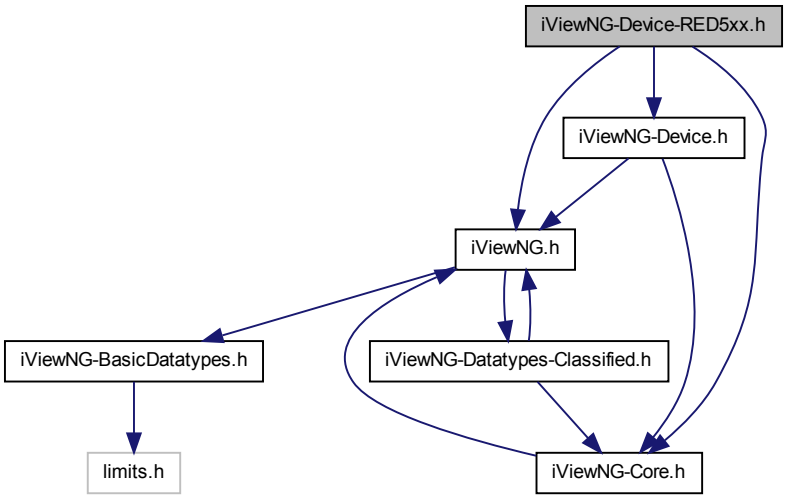
For using daylight fluorescent lightning.

4.12 iViewNG-Device-Legacy.h File Reference

4.13 iViewNG-Device-RED5xx.h File Reference

```
#include <iViewNG.h>
#include <iViewNG-Core.h>
#include <iViewNG-Device.h>
```

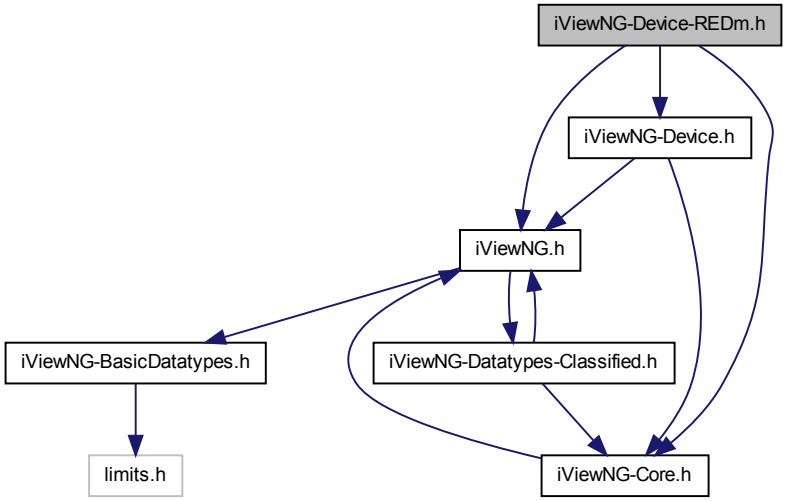
Include dependency graph for iViewNG-Device-RED5xx.h:



4.14 iViewNG-Device-REDm.h File Reference

```
#include <iViewNG.h>
#include <iViewNG-Core.h>
#include <iViewNG-Device.h>
```

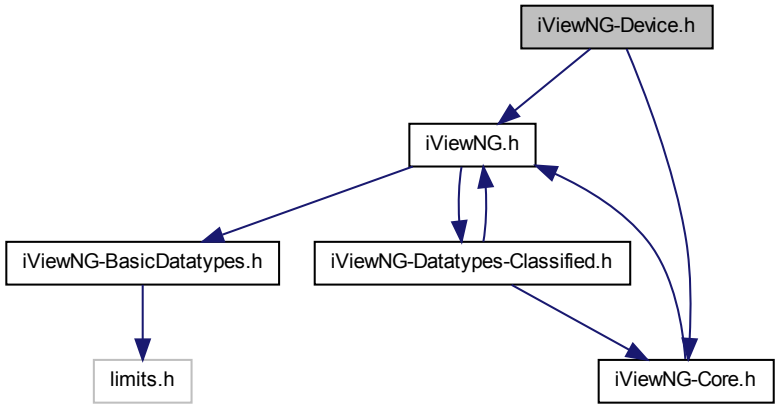
Include dependency graph for iViewNG-Device-REDm.h:



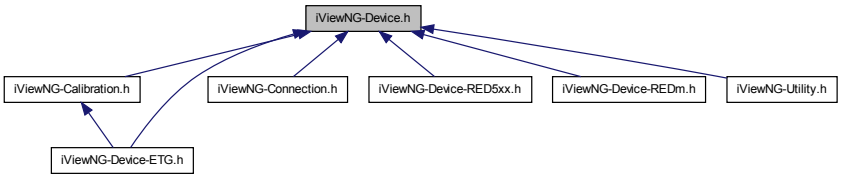
4.15 iViewNG-Device.h File Reference

```
#include <iViewNG.h>
#include <iViewNG-Core.h>
```

Include dependency graph for iViewNG-Device.h:



This graph shows which files directly or indirectly include this file:

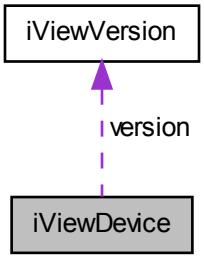


Data Structure Documentation

struct iViewDevice

Using the iView_GetDevice() function, the user may inquire which device is currently used by the iView server. The result is provided as data included into the iViewResult structure referenced by the provided iViewTicket.

Collaboration diagram for iViewDevice:



Data Fields

iViewDevice-Type	type	Identifies the device type.
iViewVersion	version	Identifies the device version.

struct iViewDeviceInfoStruct

Using iView_GetDeviceInfo(), the client application may inquire which devices are attached to a certain iViewNG server process. This data structure is returned asynchronously as answer. There may be more than just one device. In this case, the devices are linked via the 'next' element.

Data Fields

iViewDevice-Type	type	Specifies the attached device type.
iViewDevice-Info *	next	If there is another device available, this element points to the next iViewDeviceInfo data structure.
void *	versions	
void *	capabilities	Device-dependent version structure. Cast based on 'type' element, e.g. to iViewDeviceVersionEtg. Device-dependent capability structure. Cast based on 'type' element, e.g. to iViewDeviceCapabilityEtg

struct iViewDeviceParameters

Using iView_SetParameters(), the client application may define parameters for data recording. The available parameters depend on the server configuration and the iViewDeviceType. Use the iViewDeviceType to determine how to cast the void pointer.

Data Fields

iViewDevice-Type	deviceType	
void *	parameters	Defines the attached device type.

Typedef Documentation

typedef uint32_t iViewDeviceType

The iViewDevice structure contains an enumeration attribute which indicates which device is used. See iViewNG-Device-<your device>="">.h for the proper setting.

typedef uint32_t iViewTrackingMode

Datatype to encode server capabilities. Device-dependant. The operation mode(s) of the eye tracker.

typedef uint32_t iViewCameraResolution

The resolution of the submitted images.

typedef uint32_t iViewSamplingRate

Average sampling rate.

typedef uint32_t iViewWhiteBalanceProgram

Scene camera autowhitebalance program.

typedef uint32_t iViewAudioActivationState

Audio activation state.

Function Documentation

iViewRC iView_GetDeviceInfo (iViewTicket * const)

Query the iViewDeviceInfo structure.

This function produces an asynchronously delivered iViewResult data structure of the type iViewDeviceInfo.

If there is more than one device available, this data structure enumerates all available devices. If there are more than one device available, the result may be returned via the same ticket subsequently, each call containing one or more new iViewDeviceInfo structures for new device(s). Devices which are recognized after calling iView_GetDeviceInfo() will not be returned as soon as they appear.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.

The ticket progress will be set to 100% when the request was processed by the server.

Postcondition

This function produces asynchronoysly delivered iViewResult data and calls the user callback function and provides the result of the operation:

Parameters

inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a request, to indicate the state of processing the request and to provide the result data. The user is responsible for not releasing this memory unless iView_ReleaseTicket() is called to release the result data structure.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_UNSPECIFIED	on any other error

iViewRC iView_SetDeviceParameters (iViewTicket * *const*, iViewDeviceParameters const * *const*)

Set operation mode to use the DeviceType specified in DeviceParameters and provide device parameters to the iViewNG server.

Depending on the device type, a iViewResult may be submitted synchronously or asynchronously.

Precondition

The iView C API was initialized with iView_Init().
Connection to a server process was established with iView_Connect().

Postcondition

When accepted and forwarded to the iViewNG server, the ticket progress will change from 0% to 10%, the ticket state will change from IVIEWTICKETSTATUS_UNPROCESSED to IVIEWTICKETSTATUS_ACCEPTED.
This function produces asynchronoysly delivered iViewResult data and calls the user callback function to provide the result of the operation through the ticket:

- if iViewTicket.returnCode is RC_NO_ERROR:
  - iViewTicket.result is NULL
  - iViewTicket.state is IVIEWTICKETSTATUS_CLOSED
- if iViewTicket.returnCode indicated an error:
  - iViewTicket.result is NULL or a pointer to iViewServerError
  - iViewTicket.state remains at the former state

Parameters

	inout]	ticket The iViewTicket data structure, provided by the API user. iView uses the ticket structure to identify a request, to indicate the state of processing the request and to provide the result data. The user is responsible for not releasing this memory unless iView_ReleaseTicket() is called to release the result data structure.
`in`	*params*	The parameter set, depending on the device. The data structure must be allocated by the user and may be released immediately after calling this function.

Return values

RC_NO_ERROR	on success
RC_NO_INIT	if the iViewNG C API was not initialized
RC_CONNECTION_FAILED	if the connection to the server is not alive
RC_INVALID_TICKET	if ticket data structure is invalid
RC_INVALID_PARAMETER	if any parameter is invalid
RC_UNSPECIFIED	on any other error

Variable Documentation

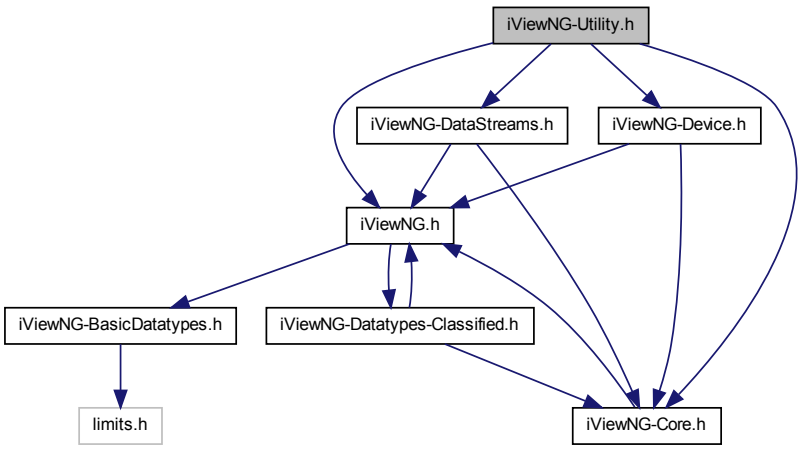
const iViewWhiteBalanceProgram IVIEWWHITEBALANCE_CURRENT = 0

Do not change the sampling rate / sampling rate unspecified.

4.16 iViewNG-Utility.h File Reference

```
#include <iViewNG.h>
#include <iViewNG-Core.h>
#include <iViewNG-Device.h>
#include <iViewNG-DataStreams.h>
```

Include dependency graph for iViewNG-Utility.h:



Function Documentation

iViewRC iView_iViewRcToString (wchar_t ** *dst*, const iViewRC *rc*)

Get a human-readable string for a given return code.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

	inout]	dst Address of pointer to hold the result. The result is allocated with malloc() and must be released by the caller. The result pointer must contain NULL.
in	*rc*	The return code to convert.

Return values

RC_NO_ERROR	on success
RC_INVALID_PARAMETER	if ticket is NULL or if the contents of ticket is not NULL
RC_OUT_OF_MEMORY	if the data structure could not be allocated
RC_UNSPECIFIED	on any other error

iViewRC iView_iViewTicketToString (wchar_t ** *readable*, iViewTicket const *const *const *ticket*)

Print the ticket contents to a char array. Useful for analyzing tickets manually.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

	inout]	readable Address of pointer to hold the result. The result is allocated with malloc() and must be released by the caller. The result pointer must contain NULL.
in	*ticket*	Ticket to convert.

Return values

RC_NO_ERROR	on success
RC_INVALID_PARAMETER	if ticket is NULL or if the contents of ticket is not NULL
RC_OUT_OF_MEMORY	if the data structure could not be allocated
RC_UNSPECIFIED	on any other error

iViewRC iView_iViewDeviceParametersToString (wchar_t ** *readable*, iViewDeviceParameters const *const *params*)

Print the DeviceParameters parameter block contents to a char array. Useful for analyzing parameters manually.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

	inout]	readable Address of pointer to hold the result. The result is allocated with malloc() and must be released by the caller. The result pointer must contain NULL.
in	*params*	Parameter block to convert.

Return values

RC_NO_ERROR	on success
RC_INVALID_PARAME-TER	if ticket is NULL or if the contents of ticket is not NULL
RC_OUT_OF_MEMORY	if the data structure could not be allocated
RC_UNSPECIFIED	on any other error

iViewRC iView_iViewDataStreamTypeToString (wchar_t ** *readable*, const iViewDataStreamType *type*)

Print the iViewDataStreamType enum to a char array. Useful for analyzing parameters manually.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

	inout]	readable Address of pointer to hold the result. The result is allocated with malloc() and must be released by the caller. The result pointer must contain NULL.
in	*type*	Parameter to convert.

Return values

RC_NO_ERROR	on success
RC_INVALID_PARAME-TER	if ticket is NULL or if the contents of ticket is not NULL
RC_OUT_OF_MEMORY	if the data structure could not be allocated
RC_UNSPECIFIED	on any other error

double iView_GetNanValue ( )

Return the value for NAN (not-a-number).

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Note

Please mind that it's not possible to compare NAN values using the '==' operator! Use iView_IsNan() instead!

Return values

NAN	as the iViewNG library encodes it.

double iView_GetInfValue ( )

Return the value for +INF (positive infinity).

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Return values

+INF	as the iViewNG library encodes it.

char iView_IsNan (const double *num*)

Test if a given variable contains NAN.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in	*num*	the variable to test

Return values

0	if 'num' does not contain NAN
1	if 'num' contains NAN Test if a given variable contains NAN.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in	*num*	the variable to test

Return values

0	if 'num' does not contain NAN
1	if 'num' contains NAN

char iView_IsInf (const double *num*)

Test if a given variable contains +INF.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in	*num*	the variable to test

Return values

0	if 'num' does not contain +INF
1	if 'num' contains +INF Test if a given variable contains +INF.

Convenience function. This function does not interact with the iViewNG server. This function does not produce asynchronous return values.

Parameters

in	*num*	the variable to test

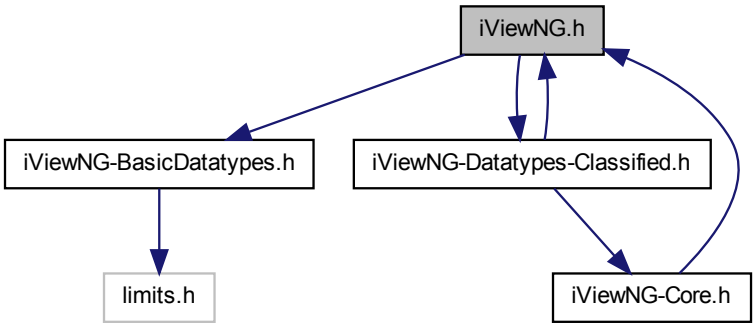
Return values

0	if 'num' does not contain +INF
1	if 'num' contains +INF

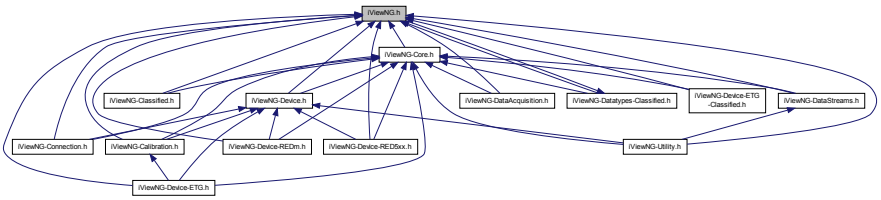
4.17 iViewNG.h File Reference

```
#include "iViewNG-BasicDatatypes.h"
#include "iViewNG-Datatypes-Classified.h"
```

Include dependency graph for iViewNG.h:



This graph shows which files directly or indirectly include this file:



Detailed Description

The C API function is provided to SMI customers and supposed to be used as well by internal software, e.g. MiiController, ExperimentCenter, BeGaze. The C API consists of data structures and a set of static DLLFUNCTION functions which are called by the user. The C API is mapped to the C++ Api class. For every static C function, there is a C++ instance method with the identical name. The C function call is mapped to the C++ method. The C API converts exceptions to return codes and provides the result data to the user application.

Macro Definition Documentation

#define DLL_DEC_PRE extern "C"

#define DLL_DEC_POST

#define DLLCALL

4.18 D:/Research/iViewNG/Implementation/SMI/source/iViewNGLib/Api/iViewNG-SDK/iViewNG-SDK-Documentation.dox File Reference

Index

DLL_DEC_POST
iViewNG.h, 90
DLL_DEC_PRE
iViewNG.h, 90
DLLCALL
iViewNG.h, 90

iViewCalibrationData, 21
iViewCalibrationParameters, 20
iViewCalibrationParametersEtg, 77
iViewDataStream, 60
iViewDataStreamEndOfStream, 60
iViewDataStreamEtgServerStatus, 75
iViewDataStreamEyeImage, 56
iViewDataStreamGazeSample, 61
iViewDataStreamH264SceneImage, 56
iViewDataStreamSceneImage, 55
iViewDataStreamSpecGazeFilter, 58
iViewDataStreamSpecQualityOfService, 59
iViewDataStreamSpecSampleRate, 58
iViewDataStreamSpecStruct, 58
iViewDataStreamTrackingStatus, 57
iViewDevice, 81
iViewDeviceCapabilityEtg, 74
iViewDeviceInfoStruct, 81
iViewDeviceParameters, 82
iViewDeviceParametersEtgCameraPlayback, 75
iViewDeviceParametersEtgIdfPlayback, 68
iViewDeviceParametersEtgIdfRecalibrationPlayback, 69
iViewDeviceParametersEtgRuRealtime, 77
iViewDeviceParametersEtgRuVideoPlayback, 70
iViewDeviceParametersEtgVideoPlayback, 67
iViewDeviceVersionEtg, 75
iViewEyeSample, 60
iViewGazeCorrectionParameters, 70
iViewHost, 30
iViewHostAddress, 29
iViewHostAddress.ipAddress, 31
iViewHostListStruct, 31
iViewIdfPlaybackData, 66
iViewImage, 54
iViewOverlaySpec, 36
iViewOverlaySpecListStruct, 37
iViewResultStruct, 36
iViewServerError, 35
iViewStreamSubscription, 59
iViewTicket, 37
iViewTime, 35
iViewVector2D, 35
iViewVector3D, 36
iViewVersion, 35
IVIEW_CALLBACK
iViewNG-Core.h, 40
iView_AddLicenseKey
iViewNG-Core.h, 44
iView_Calibration_Cancel
iViewNG-Calibration.h, 25
iView_Calibration_SendData
iViewNG-Calibration.h, 24
iView_Calibration_Setup
iViewNG-Calibration.h, 21
iView_Calibration_Start
iViewNG-Calibration.h, 23
iView_CloseLogfile
iViewNG-Classified.h, 28
iView_Connect
iViewNG-Connection.h, 32
iView_CreateTicket
iViewNG-Core.h, 43
iView_Disconnect
iViewNG-Connection.h, 33
iView_GetCallback
iViewNG-Core.h, 43
iView_GetDeviceInfo
iViewNG-Device.h, 83
iView_GetExperimentData

iViewNG-Device-ETG-Classified.h, 71
iView_GetGazeCorrectionData
iViewNG-Device-ETG-Classified.h, 73
iView_GetInfValue
iViewNG-Utility.h, 88
iView_GetLibraryVersion
iViewNG-Core.h, 40
iView_GetLocalTime
iViewNG-Core.h, 47
iView_GetNanValue
iViewNG-Utility.h, 87
iView_GetParticipantData
iViewNG-Device-ETG-Classified.h, 71
iView_GetServerTime
iViewNG-Core.h, 45
iView_Init
iViewNG-Core.h, 40
iView_IsInf
iViewNG-Utility.h, 89
iView_IsNan
iViewNG-Utility.h, 88
iView_KillServer
iViewNG-Classified.h, 28
iView_LogString
iViewNG-Classified.h, 27
iView_Recording_Start
iViewNG-DataAcquisition.h, 50
iView_Recording_Stop
iViewNG-DataAcquisition.h, 51
iView_ReleaseResult
iViewNG-Core.h, 44
iView_ReleaseTicket
iViewNG-Core.h, 43
iView_SetCallback
iViewNG-Core.h, 42
iView_SetDeviceParameters
iViewNG-Device.h, 84
iView_SetExperimentData
iViewNG-Device-ETG-Classified.h, 71
iView_SetGazeCorrectionData
iViewNG-Device-ETG-Classified.h, 72
iView_SetParticipantData
iViewNG-Device-ETG-Classified.h, 71
iView_Shutdown
iViewNG-Core.h, 41
iView_ShutdownServer
iViewNG-Core.h, 42
iView_StartDataAcquisition
iViewNG-DataAcquisition.h, 48
iView_StartServer
iViewNG-Core.h, 41
iView_StopDataAcquisition
iViewNG-DataAcquisition.h, 49
iView_SubscribeDataStream
iViewNG-DataStreams.h, 63
iView_UnsubscribeDataStream
iViewNG-DataStreams.h, 64
iView_iViewDataStreamTypeToString
iViewNG-Utility.h, 87
iView_iViewDeviceParametersToString
iViewNG-Utility.h, 86
iView_iViewRcToString
iViewNG-Utility.h, 85
iView_iViewTicketToString
iViewNG-Utility.h, 86
iViewAudioActivationState
iViewNG-Device.h, 83
iViewCalibrationType
iViewNG-Calibration.h, 21
iViewCameraResolution
iViewNG-Device.h, 82
iViewDataStreamSpecType
iViewNG-DataStreams.h, 63
iViewDataStreamState
iViewNG-DataStreams.h, 63
iViewDataStreamType
iViewNG-DataStreams.h, 62
iViewDeviceType
iViewNG-Device.h, 82
iViewEyeIdentifier
iViewNG-DataStreams.h, 62
iViewLocationScope
iViewNG-Connection.h, 32
iViewNG-BasicDatatypes.h, 18
int16_t, 19
int32_t, 19
int64_t, 19
int8_t, 19
UINT_MAX, 18
uint16_t, 19

uint32_t, 19
uint64_t, 19
uint8_t, 19
iViewNG-Calibration.h, 19
iView_Calibration_Cancel, 25
iView_Calibration_SendData, 24
iView_Calibration_Setup, 21
iView_Calibration_Start, 23
iViewCalibrationType, 21
iViewNG-Classified.h, 27
iView_CloseLogfile, 28
iView_KillServer, 28
iView_LogString, 27
iViewNG-Connection.h, 29
iView_Connect, 32
iView_Disconnect, 33
iViewLocationScope, 32
iViewServerAddressType, 32
iViewNG-Convenience.h, 34
iViewNG-Core.h, 34
IVIEW_CALLBACK, 40
iView_AddLicenseKey, 44
iView_CreateTicket, 43
iView_GetCallback, 43
iView_GetLibraryVersion, 40
iView_GetLocalTime, 47
iView_GetServerTime, 45
iView_Init, 40
iView_ReleaseResult, 44
iView_ReleaseTicket, 43
iView_SetCallback, 42
iView_Shutdown, 41
iView_ShutdownServer, 42
iView_StartServer, 41
iViewOverlayOption, 39
iViewOverlayPosition, 39
iViewOverlayType, 39
iViewRC, 38
iViewResultType, 39
iViewSdkType, 38
iViewTicketNumber, 39
iViewTicketProgress, 39
iViewTicketState, 39
iViewNG-DataAcquisition.h, 48
iView_Recording_Start, 50
iView_Recording_Stop, 51
iView_StartDataAcquisition, 48
iView_StopDataAcquisition, 49
iViewNG-DataStreams.h, 53
iView_SubscribeDataStream, 63
iView_UnsubscribeDataStream, 64
iViewDataStreamSpecType, 63
iViewDataStreamState, 63
iViewDataStreamType, 62
iViewEyeIdentifier, 62
iViewQualityOfService, 63
iViewNG-Datatypes-Classified.h, 65
iViewNG-Device-ETG-Classified.h, 66
iView_GetExperimentData, 71
iView_GetGazeCorrectionData, 73
iView_GetParticipantData, 71
iView_SetExperimentData, 71
iView_SetGazeCorrectionData, 72
iView_SetParticipantData, 71
iViewNG-Device-ETG.h, 74
iViewNG-Device-Legacy.h, 79
iViewNG-Device-RED5xx.h, 79
iViewNG-Device-REDm.h, 79
iViewNG-Device.h, 80
iView_GetDeviceInfo, 83
iView_SetDeviceParameters, 84
iViewAudioActivationState, 83
iViewCameraResolution, 82
iViewDeviceType, 82
iViewSamplingRate, 82
iViewTrackingMode, 82
iViewWhiteBalanceProgram, 82
iViewNG-Utility.h, 85
iView_GetInfValue, 88
iView_GetNanValue, 87
iView_IsInf, 89
iView_IsNan, 88
iView_iViewDataStreamTypeToString, 87
iView_iViewDeviceParametersToString, 86
iView_iViewRcToString, 85
iView_iViewTicketToString, 86
iViewNG.h, 89
DLL_DEC_POST, 90
DLL_DEC_PRE, 90
DLLCALL, 90

iViewOverlayOption
iViewNG-Core.h, 39
iViewOverlayPosition
iViewNG-Core.h, 39
iViewOverlayType
iViewNG-Core.h, 39
iViewQualityOfService
iViewNG-DataStreams.h, 63
iViewRC
iViewNG-Core.h, 38
iViewResultType
iViewNG-Core.h, 39
iViewSamplingRate
iViewNG-Device.h, 82
iViewSdkType
iViewNG-Core.h, 38
iViewServerAddressType
iViewNG-Connection.h, 32
iViewTicketNumber
iViewNG-Core.h, 39
iViewTicketProgress
iViewNG-Core.h, 39
iViewTicketState
iViewNG-Core.h, 39
iViewTrackingMode
iViewNG-Device.h, 82
iViewWhiteBalanceProgram
iViewNG-Device.h, 82
int16_t
iViewNG-BasicDatatypes.h, 19
int32_t
iViewNG-BasicDatatypes.h, 19
int64_t
iViewNG-BasicDatatypes.h, 19
int8_t
iViewNG-BasicDatatypes.h, 19

UINT_MAX
iViewNG-BasicDatatypes.h, 18
uint16_t
iViewNG-BasicDatatypes.h, 19
uint32_t
iViewNG-BasicDatatypes.h, 19
uint64_t
iViewNG-BasicDatatypes.h, 19
uint8_t
iViewNG-BasicDatatypes.h, 19